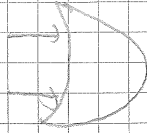
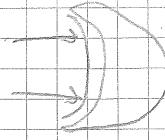


AND



OR



XOR



NOT

$$(\text{---})_{16} = (\text{---})_2$$

$$(\text{---})_8 = (\text{---})_2$$

AND $\rightarrow$ '✓✓'yu tespit eder.

OR $\rightarrow$ X.X'ı tespit eder.

XOR $\rightarrow$ Farklıları tespit eder. (Hamming Distance)



online.yildiz → yükle

Yarın ilk ödev online → 4 h 10 soru / bu kolay

• İnti. Senin, kullan 16-18 → 1 saat ayık

Number Systems			
$(\sim)_2$	$(\sim)_8$	$(\sim)_{10}$	$(\sim)_{16}$
① <u>Binary</u>	② <u>Octal</u>	③ <u>Decimal</u>	④ <u>Hexadecimal</u>
0's and 1's	0	0	0 A (10)
① 0000	1	1	1 B (11)
0001	2	2	2 C (12)
0010	3	3	3 D (13)
⋮	⋮	⋮	⋮
1111	7	7	7 F (15)

① A B C D
16'lik olduğunu gösterir.

0x ABCD = (ABCD)₂ → Her bas. 4 bas.

• 0x ABCD = 1010 1100 1100 1101

• 0x (F)₁₆ = (1111)₂

* Bellek adreslemeye

hexadı kullanırız.

1	1	1	1	1	1	1	1
7	6	5	4	3	2	1	0

→ Belleğe yazacağın en k. sayı → 1 byte

8' byte 1 → 256

256 farklı bilgi

* Herisi 1 → tek

→ 0x 1000
→ 0x 1001
→ 0x 1002
→ 0x 1003
⋮
→ 0x 1009
→ 0x 100A

2 byte → Maks = 2¹⁶ - 1

Farklı Sayı = 2¹⁶

512 - için 2 byte → 1 word

Memory



(Hesap mak. kullanmay kendin yap)

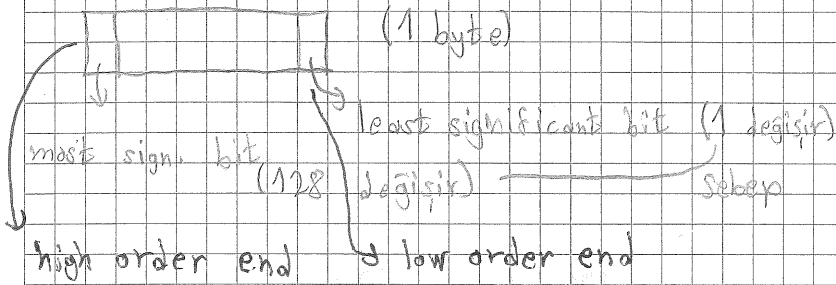
(ödevdeki soruları çöz.lü yap)

$$(13)_{10} = (11)_{10}$$

$$(1101)_2$$

$$\begin{array}{r} 13 \overline{) 2} \\ \underline{13} 2 \\ 1 2 \\ \underline{10} 2 \\ 1 2 \\ \underline{10} 2 \\ 1 2 \\ \underline{10} 2 \\ 1 2 \end{array}$$

(2'ye bölme)
diğerinde bitti.



* Bellek üstünde işiende 8, 16 gibi 8'in katı bitlerle ifade yaz.

10 bitle yazdığın sayıyı 16 bitle yaz. Baş 0'lar.

Byte'arla yazıyoruz çünkü. Gereksiz byte olmasın.

(Signed Numbers)
Negative Numbers

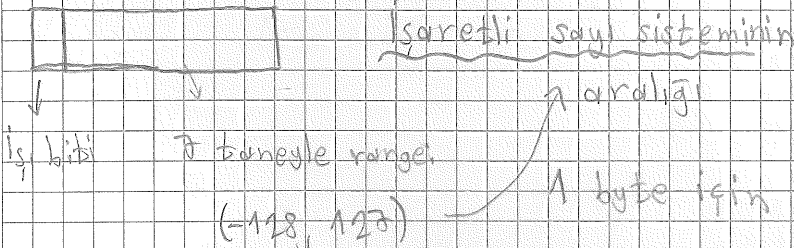
* 1 yerine 0, 0 yerine 1 yaz

Sayılar 1'e Tömlenen

İşaret Biti → En anlamlı olan

$$\begin{array}{r} 1024 \\ \times 64 \\ \hline 65536 \\ \times 64 \\ \hline 4194304 \end{array}$$

(bil)
2 byte maks
65,535



0 1
↓ ↓
+ -

① 2'nin Tümlöyeni (complement)

01000000

10111111

11000000

00000000 → 0

01111111 → 127

10000000 → -128

11111111 → -1

(-1)er saklamamın 2'gözü

② Excess Notation

4 bit → (-8, 7)

Bit Pattern	Value Represented
1111	7 (15-8)
1110	6 (14-8)
1001	1
1000	0
0111	-1 (7)
0110	-2 (6)
0101	-3 (5)

① → +

② → -

* 1'in Tümlü
-1'erde kullanırsan

2 tane 0 olur.

2'nin Tümlü kullan

127

01111111

-1'ler, 2'nin Tümlöyeni al.

10000000 önüne
01111111 - kay

+ 10000000 → 128

Toplamamın 2 olunca elde olup
Yolu: Sola aktarıp 0 olur.

* 1'e başlayıp
kalanları 0 ise → 0'dır.

• Excess 8 Notation (4 bit)

• Excess 128 Notation (8 bit)

Storing Fractions → Yine . kullanıyorsun

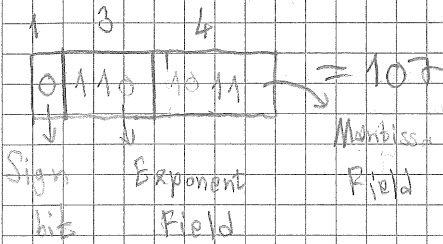
$$\frac{3}{8}, 0.375$$

$$101.110$$

$2^2 \ 2^1 \ 2^0 \ 2^{-1} \ 2^{-2} \ 2^{-3}$

5

Floating Point Notation



* Detaylı sayılar için daha çok bit (alan) lazım.
Küçük sayılar için

Exponent

$$110 \rightarrow 6 - 4 = 2 \text{ (} 2^2 \text{, } \rightarrow \text{)}$$

$$+ 110.1011$$

$$+ 1011$$

* (-4, 3) ise
sola 4, sağa 3
kaydırabilir.

$$10.11 = 2.75$$

$$\begin{array}{r} 4 \times \\ 62 \\ 8 \\ 2 \\ 1 \end{array} \begin{array}{l} 1011 \\ 1011 \\ 1011 \\ 1011 \\ 1011 \end{array}$$

$$11 \ 00 \ 1111 \ 00$$

Excess Mantığı

$$+ 011.1100$$

$$011 \rightarrow 3 - 4 = -1 \text{ (1, } \leftarrow \text{)}$$

$$+ \sim 1100 \rightarrow .01100 = \frac{3}{8} = 0.375$$

Kaydırırken
sadece ondalığa

* Noktayı kaydırmamak seçenek!

100

$$11 - \frac{5}{8} \rightarrow \frac{77}{8}$$

$$-0.625$$

$$\frac{1}{2} + \frac{1}{8}$$

$$\cdot 101$$

olmalı

$$4 - k \geq 0$$

$$1 \overline{1010} \checkmark$$

$$1 \overline{101} \times$$

$$5 - k \geq 1$$

101 olmalı

$$1 \overline{100.1010} \rightarrow -\frac{5}{8}$$

* Noktadan sonra
görülen ilk 1'den
itibaren yaz.

$$11 \frac{3}{8} \rightarrow 2.375$$

$$\frac{1}{4} + \frac{1}{8}$$

$$\frac{0}{1} \overline{011.1100}$$

1 sola gitmeli

$$3 - k \geq -1$$

Truncation Error

* Noktanın solunda da 1 varsa
öncelik onundur.

$$\frac{2 \frac{5}{8}}{1} = 2.625$$

$$110.1010 = 2 \frac{1}{2}$$

$$4 - k \geq 2$$

float $\rightarrow 4$ bit

double $\rightarrow 8$

* Overflow

$$\overline{11111111}$$

8 bit 511

$\downarrow$

8 bit 255

Unsigned

Signed

* Alan yoksa
saklamayız mı?

Araştırın.



Single Precision Floating Point

-32 bit $\rightarrow$ 1 + 8 + 23
Sign exponent mant.
f.

10^{38} , 10^{-37}

7 decimal

Double Pr. Fl. Po. $\rightarrow$ -64 bit, 15 decimal

Data Storage

veri $\leftrightarrow$ bilgi $\leftrightarrow$ data
Data $\leftrightarrow$ Information $\leftrightarrow$ bilgi
bilgi daha anlamlı, yapılmış

- Character
- Image
- Text
- Video
- Audio
- Webpage
- Program
- Word
- Excel
- Files
- CPU Instructions
- Numb.
- Fictional numb.
- Letters
- Colors

Images

Pixel: Picture Element

height

Full HD: 1920 x 1080 pixel

width

Siyah-beyaz

1 ve 0

1 byte için

0 ve 255

gri için arası $\rightarrow$ 256

Sound (Audio Data)

MP3 $\rightarrow$ Sesin sık. hal



Değeri olmayan her $\rightarrow$ Bayt tasarrufu genişlik

Text

Characters $\rightarrow$ Letters, punctuations, symbols

ASCII Table

Covers all
of them

American Stand. for.

Information Interchange

$$\text{'c' - 'a' = 2}$$

ASCII Table

0
numbers
letters
symbols
255

1 byte
kullanımı
(256 karakter)

• Rakam ve harfler sıralı

A → 65 } Ezbere
O → 48 } gerek
yok.

UTF-8 Code

2 byte'ya kullanıyor.

B → 65

C → 66

⋮

Mb: Sıkıştırma

Alan kazan

Hişsedilir.

1) Kayıpsız → orijinal dönüş var.

2) Kayıplı → Dönüş yok

Duyamadığın (ses ve görüntü)
frekanslarıdır.

• Görülür fark az.
gerek var.

* Karakterler

Full HD yobiyorsa 8K'ya ne

'a', 'b' (tek bir nokta)

• String → "hello world"
(character)

• Sayılar → 3, 5, 7

Kayıplı'da encode, decode ayırtmasını
vermez.

• Encoding: Kodlama

• Decoding: Eski haline
sıkıştırma kazandırma
ve güvenlik

Run-Length Encoding

AAA BBCCDD E F F → 13 byte

3A 2B 3C 2D E 2F → 12 byte

• Tekrar eden bilgi

AAAA AAAAAA BB → 12 byte

arttıkça verim artar.

10A 2B → 4 byte

10 → 1 byte

En verimli

Veriye

0 ≤ x ≤ 255

okış. yolu

bağlı



Frequency Dependent Encoding

- Huffman coding

→ az

• Sık kullanılan az bit kullan. → Tasarruf

- Huffman coding
düşüktür

→ f_j, j, g

Communication Errors

→ Bir bilgisayardaki bütün birimler arasındadır.

- Error Detection

- Error Correction

Daha çok iş

Hamming Distance (Uzaklık)

$\begin{bmatrix} 0100 \\ 1000 \end{bmatrix}$ Hd. Di
= 2

(2 bit değişmiş)

XOR kullanarak tespit edilebilir.
Farklıları tespit eder.

pattern

Hamming Di. between two bits is the number of bits which the pattern differs.

- Error Detection

↓
Parity bit

→ Additional bits

Odd or even

number of 1s = even

number of 1s = odd

1 0 1 0 0 0 0 1

↓
Parity bit

0 0 1 0 0 0 1 1 0

* Total pattern has an odd number of 1s.

- even



* Her işin maliyeti var.

* Tek bitlik değişimleri tespit eder.

→ Daha çok tespit → Daha çok bedel.

Error Correcting Codes

<u>Symbol</u>	<u>Code</u>	<u>Pattern Received</u>	<u>Ham Di</u>
A	000000	010 100	2
B	001111	(hepsi)	4
C	010011		3
D	011100		1
E	100110		3
F	101001		5
G	110101		2
H	111010		4

En yakın olan
↓
İhtimalce doğru

Ham Di → en az 3

• İki tane aynı çıkarsa ne olur?

• Çok bit değişirse sistem çöker. 1 ve 2 bit değişimi için kullanılır ve güvenilirdir.

* Checkbyte: A collection

of parity bits.

Cyclic Redundance:?

Checksum:?

RAID Diskin belirli bir alanını veri koruma için ayırır.

Analog veriyi etkilemek → Dijital veride kolaydır.

...../...../.....
tekrar yapmaya olanak sağlar.

Algoritma

Her türlü doğru girdi için her türlü doğru çıktı oluşturur.

*An algorithm is an ordered set of unambiguous, executable steps that defines a terminating step.

*Harezmi → Ortaçağ atığı

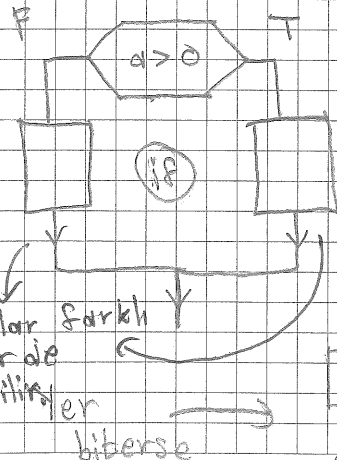
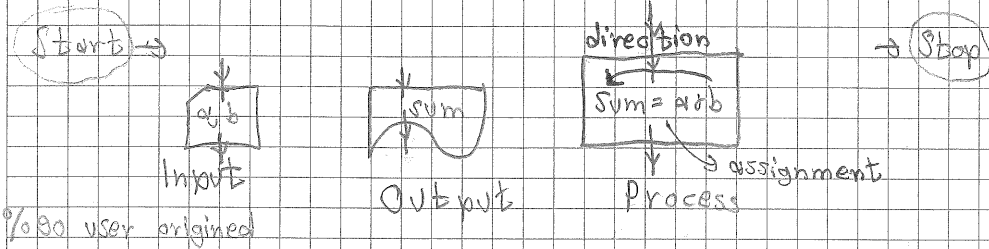
→ Oral Presentation

* → Flow Chart (Akış Diyagramı) → Misperden görsel

→ Pseudo code (Pöze kod)

→ Programming Language (C, Java, ...)

Flow Chart Notations → Start, Stop; not more

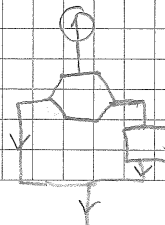


*True code must exist.

False code doesn't have to.

→ (Shape matters)

-False - kullanmayacaksan gizlemeyebilirsin.



Gök kullanma daha iyi



Assignment Operation

$$a = b$$

$$c = 5$$

$$a = a + b$$

eski a
kayıp

(a = a diğip
saklaya
bilirdin)

Arithmetic Operator

+

-

*

/

mod

DIV

tam sayı
bölmesi

iki
operatör
alınır.

>

<

>=

<=

=

≠

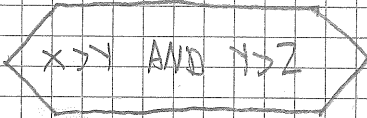
Relational (Comparison) Operators

Logic Operators

AND

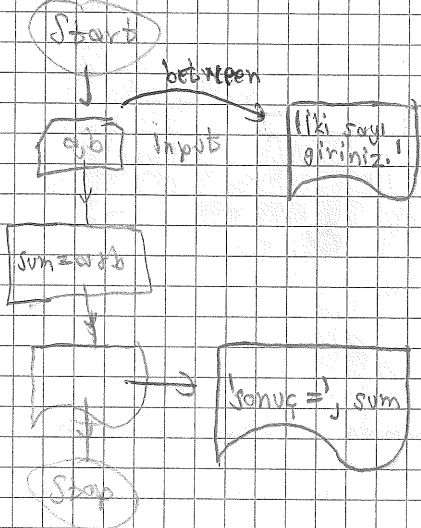
OR

...



* Karşılaştırmaları
iki'li iki'li yap

II 2 sayıyı toplayan algoritmanın akış diyagramını çiziniz.



Analizi: Her aşamada değişkenlerin değerlerini yaz.

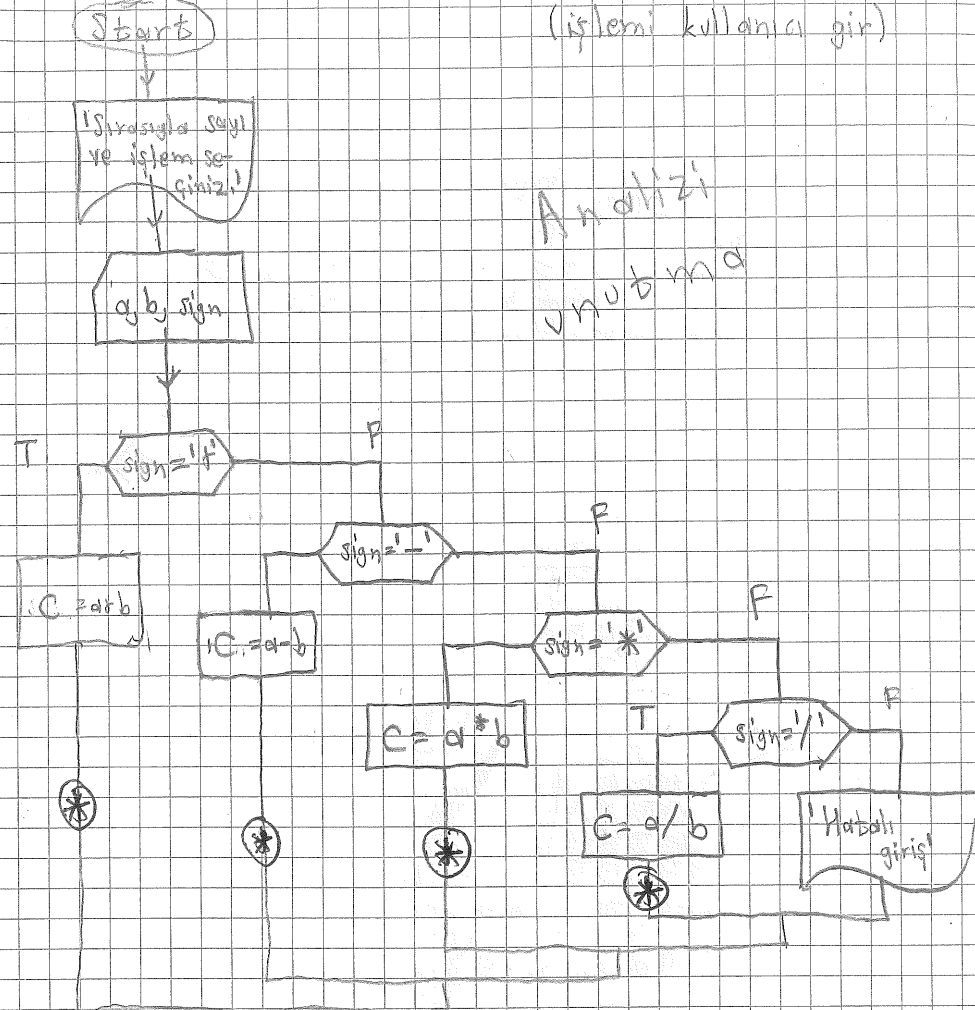
	a	b	sum
①	5	3	8
②	3	-4	-1

* Değişken isimlerini
anlamlı seç.

1) İşlem yapabilen algoritma akış diyagramını çiz

(işlemi kullanıcı gir)

Analiz
ünütüme



* İşlem
bölmeysen
 $b=0$ 'i
kontrol
et.

Bu durumda 'Hatalı giriş' ben
sonra da c'yi yazmaya çalışıyor.
Output kısmını her işlemten
sonraya (*) al.



Flowgorithm

Lucidchart

|| Başlangıç ve bitiş verilen iki doğru parçasının paralel olup olmadığını bulan bir algoritma yaz. (Ak. di. ile)

(Koord. düzl. de)

Start

'Noktaları belirleiniz.'

$x_1, y_1, x_2, y_2,$
 x_3, y_3, x_4, y_4

$a \neq b \neq c$
Yapılamaz.
İşlemler her zaman ikili.

T AND F
 $x_1 = x_2$ AND $x_3 = x_4$

'Doğrular paralel'

T OR F
 $x_1 = x_2$ OR $x_3 = x_4$

'Paralel değil.'

$m_1 = (y_2 - y_1) / (x_2 - x_1)$
 $m_2 = (y_4 - y_3) / (x_4 - x_3)$

P T
 $m_1 = m_2$

① yap

② kontrol et.

Mod
DIV

Verilen sayının
akış diy. çiziniz!

Armstrong sayı olup dlm. bulan alg. nın

Arma. num.

$$abcde = a^n + b^n + c^n + d^n + e^n$$

$n \rightarrow$ number of digits $n=3$

$$abc = a^3 + b^3 + c^3$$

$$abc \text{ DIV } 10 = ab$$

Start

3 baslık bir
sayı giriniz!

X

bir = $X \bmod 10$
on = $(X \text{ DIV } 10) \bmod 10$
yüz = $\text{sayı DIV } 100$

Analiz:

X	bir	on	yüz	tmp	Çıktı
100	1	0	0	1	De.
153	3	5	1	153	Arm.

farklı durumların
çalıştığını göster.

tmp = bir.bir.bir + on.on.on + yüz.yüz.yüz

F

X = tmp

T

'Değildir.'

'Armstrong
sayısı.'

Stop

işlemleri işle

sokabilirsin ama

işlemin varsa ve

özellikle uzunsa

işlem bloğunda yap.

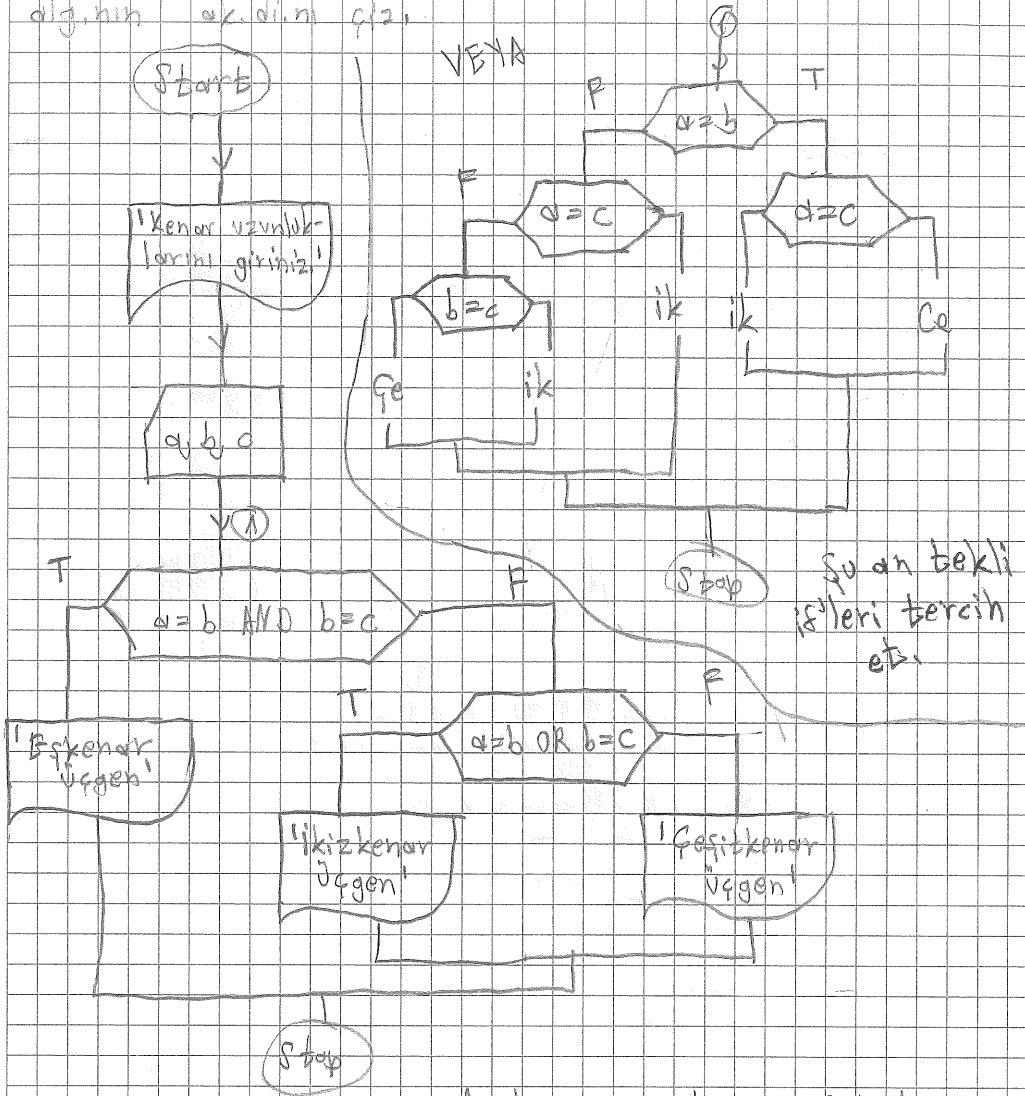


DOĞUŞ
ÜNİVERSİTESİ

dogus.edu.tr | 444 79 97 | info@dogus.edu.tr

[illegible]

11) Kenar 12,101 verilen bir üçgenin çevresini yazdırarak
dij. nın ek. d. nı çiz. ④



Analizini her
outputs için
yaz.

Analyz:	<u>a</u>	<u>b</u>	<u>c</u>	<u>Output</u>
	3	3	5	
	4	4	4	
	3	4	5	
	4	5	4	
	3	4	4	

Gereksiz iş yapma.

Döngüler/ Loops

- For loop

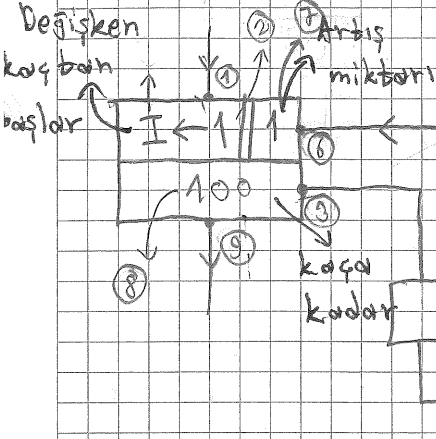
- While loop

↳ do...while

↳ while

↳ repeat until

For Loop



* Son değer dahil.

Döngünün içinde işlem, if döngü olabilir.

* Döngüye girerken

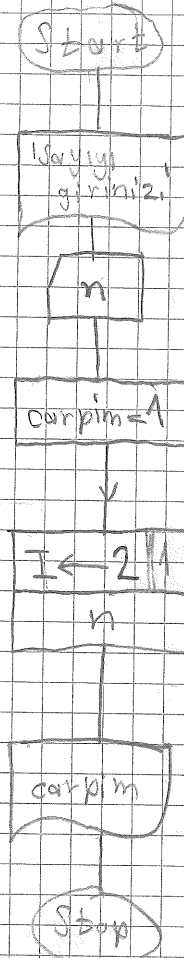
$I \leq 100$ mü diye bakar.

For Loop

Belirli sayıda adım

$$0! = 1$$

1) Verilen n 'in fakt. ini hesap. alg. eklediğiz.



Analiz:

	N	$carpim$	I
$0!$	1	1	1
$1!$	1	1	1
$2!$	2	1	1

1
2
3

Çıkarken
 $I = 3$ tür.

olsa da olur, olmaya da
olursa daha iyi

* 0 döngüye girmez, 1 çıkar. 0 zaman 0 ve 1
* I 2'den başlamalı, 1'e girme gereksiz.

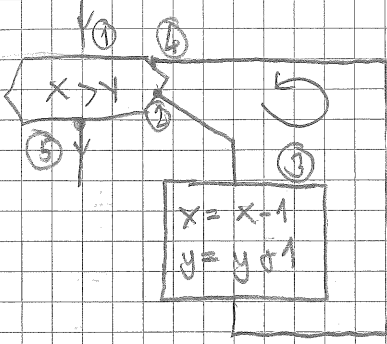
* Döngüde tam sayılar
olmalı.
* I yerine her şey denebilir.
Genelde I, J, K konur.



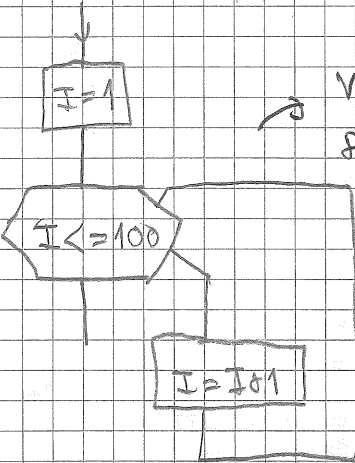
Adım Sayısı belirsiz

While Loop

Meal: X, Y den küçük olduğu sürece yap.



*



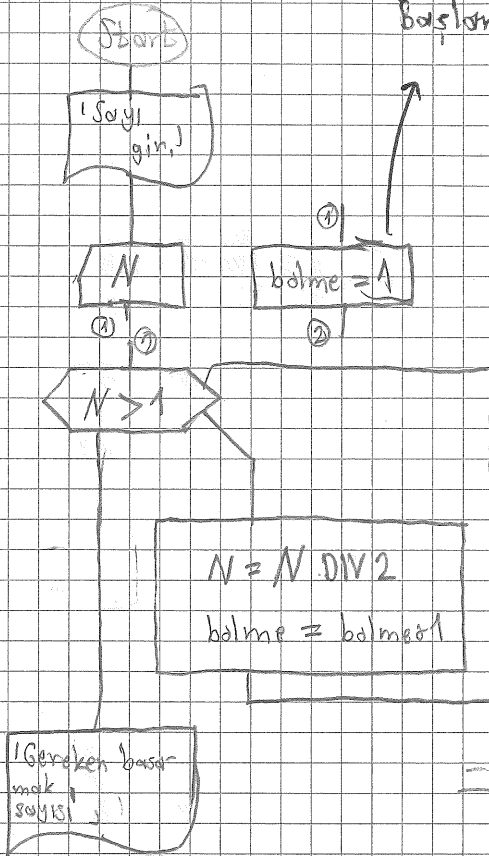
While ile
for döngüsü

Analizi hep sadece
algoritmaya göre yap.

Tasarımını hep süstümlü minimum

olacak şekilde yap.

N Verilen N için ikilik sayı olarak kaç basamak ihtiyacı old. hes. yapı. ol. di. piziniz.



Başlangıç değerlerini döngüden önce belirle. (ilklendirme)

58
29
14
7
3
1

1
2
3
4
5
6

58 | 2
= 0
29 | 2
= 1
14 | 2
= 0
7 | 2
= 1
3 | 2
= 1
1 | 2
= 1

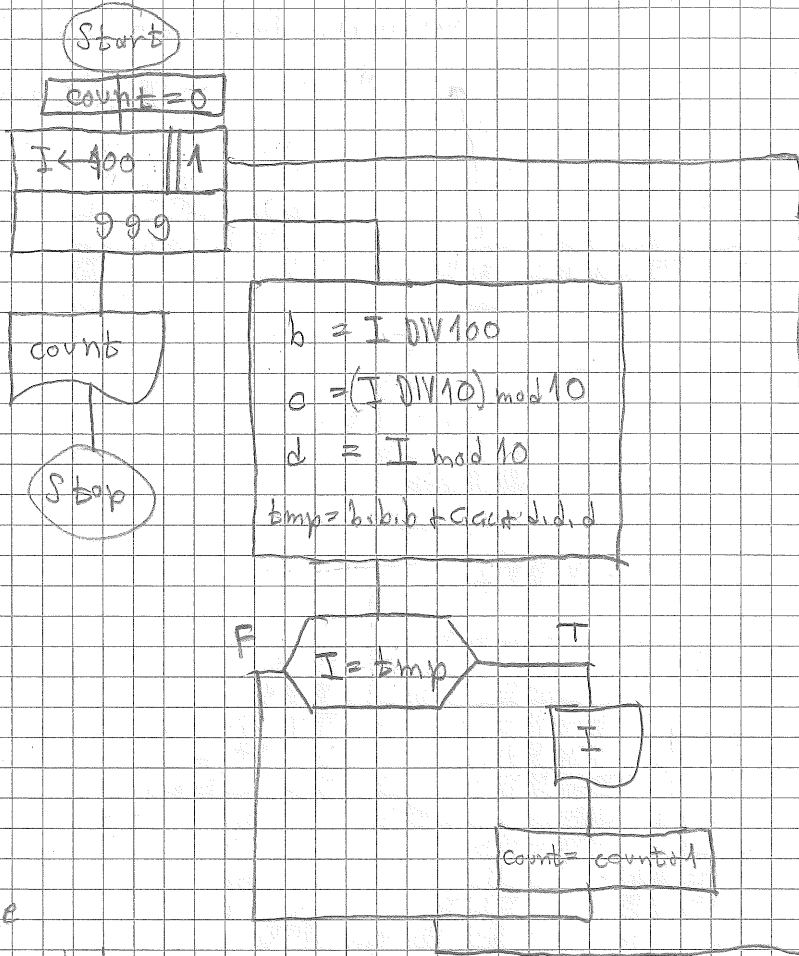
N	bilme
12	1
6	2
3	3
1	4

Analiz

Reusability

*

11 3 bas.lı Arms. sayıları bulan alg.ın akış. şiz.
Hem aradı, hem sayıları bulmalı.



*İç içe

for ve while

kullanabilirsiniz.

Çizimi doğru yap.

Neyi ne zaman

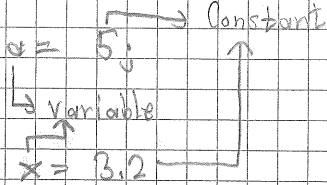
kullanacağını bil.

*Normalde I'ları bir dizide (array) saklayabilirsiniz.

- Mümkün olduğunca az not gerekmeli. ⁷ Öze sinavda
- Dizi, sabit, değişken, sorular ve C dili

Arrays, Constants, Variables

→ Array



5	1	4	0	-4	8	9
0	1	2	3	4	5	6

~~3.2 = a~~

c = b + a

~~a + b = c~~

d = b * c

1.	N.
0	N-1

dizi[6] = 9

dizi[1] = 1

dizi[0] = 5

~~dizi[7] yok~~

* Bir dizide aynı tip veriler saklanır.

değişken olabilir.

Memory

*
Adress

* Dizi elemanları bellekte arka arkaya

a	5	0x 0008
x	3.2	0x 000C
dizi[0]	5	0x 0010
dizi[1]	1	0x 0014
[2]	4	0018
[3]	0	001C
[4]	-4	0020
⋮	⋮	

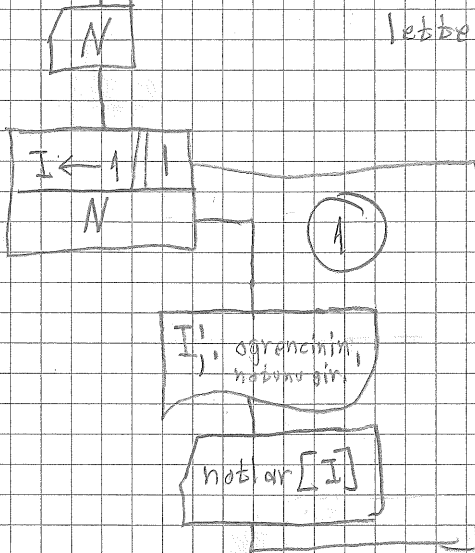
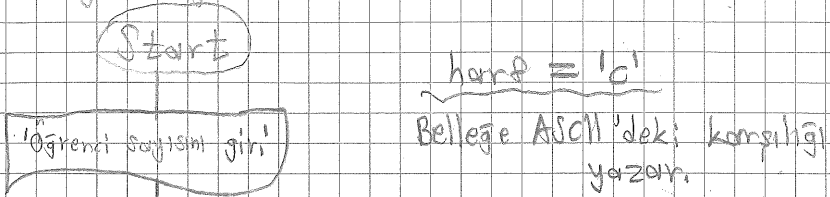
Günkü int 4 byte yer kaplar.

⚡
Bloklar

* Değişkenler dağılır

ASCII 48 → '0'

* belirli sayıda öğrencinin notu okunacak



harf = 'c'

harf = '0'

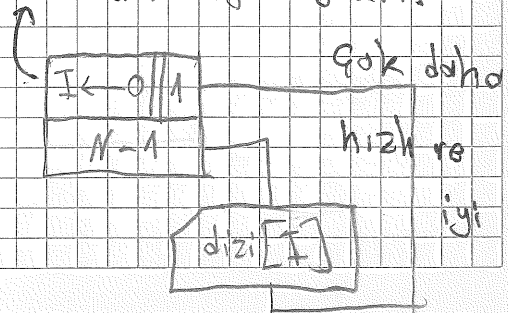
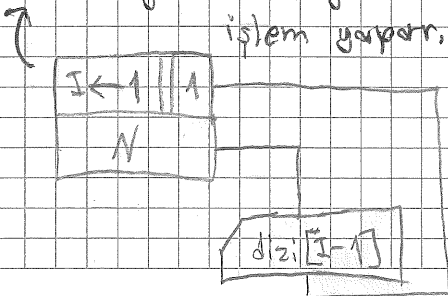
letter = '50'

① Doğru yapı.

② Hızlı ve öz yapı.

N milyonsa milyon kere işlem yapar.

Direkt değeri yazar.



DOĞUŞ
ÜNİVERSİTESİ

dogus.edu.tr | 444 79 97 | info@dogus.edu.tr

11 N adet öğrencili sınıfta ort. albi b. 5 r2, vrbü k. 1 dır.
Başarısız km. bulan alg. ak. dı. gız.

Start

"Öğrenci sayısını gir."

N

I ← 1 // 1
N

toplam = 0

I ← 1 // 1
N

I, öğrencinin notunu
girisiz.

notlar[I]

toplam = notlar[1] + toplam

1

ort = toplam / N

Stop

I ← 1 // 1
N

notlar[I] < ort

P

I

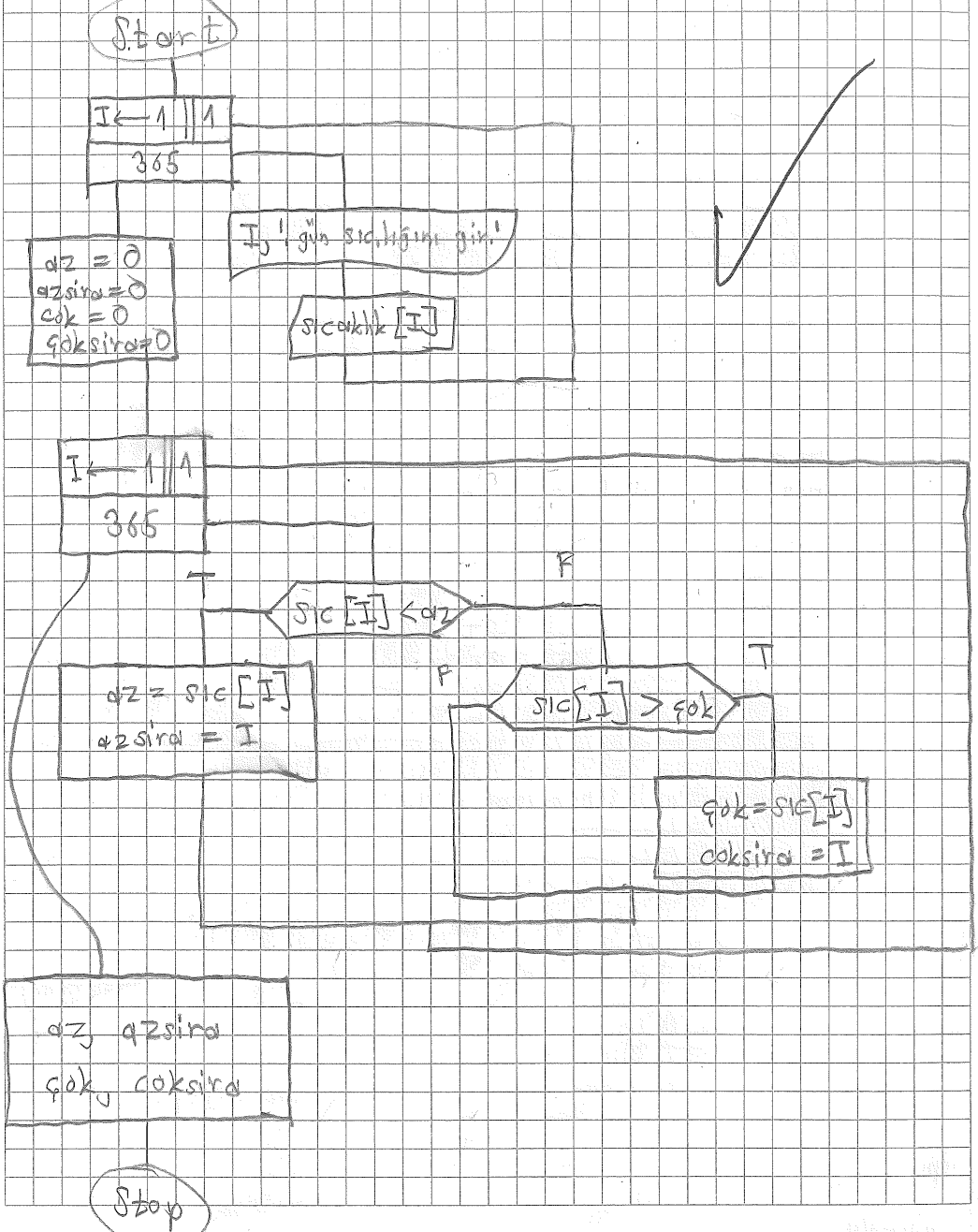
biyi float
olurdu.

Strayı
yazdırdı.

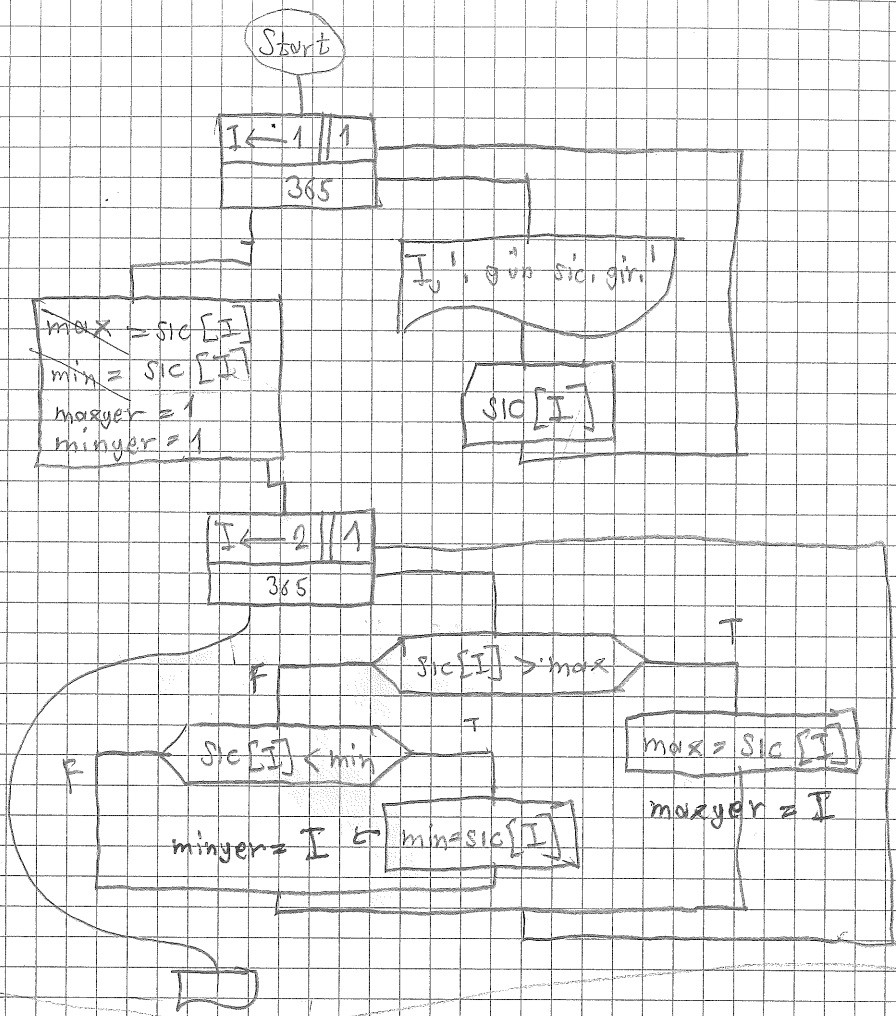


* Min - Maks *

\\ Bir yerin 365 günlük sic. bilg. verilir. En sıcak ve soğuk gün ve sic. larını yazdıran alg, ak. di. fiz.



* Her şeyi ilk for'da yapabilirsin
ama böyle daha okunur.



* kısaltmak için min, max 'ı aynı "yer"lerle iş yapma

$s[i] > s[maxyer]$ ve $s[i] < s[minyer]$ 'e

bak. Sırasıyla yerleri yazdır. (Sıra)

Son olarak $s[yer]$ 'leri yazdır. (Değerler)

C Programlama Dili

- 1970'ler
- Erişme gücü
- Hızlı
- Hata kullanılır.
- Derleyici dil (Compiler var) → Yazım hatası bildirir.
- IDE → yaz, derle, debug
- Text yaz → Terminalde > gcc ile derle.
S. / dosyaadı

* Void → Bir şey döndürmez

• ANSI C Standards → Her platformda çalışır.

include <stdio.h>

int main() {

printf ("Ögr. sayısını gir.");

scanf ("%d", &n);

printf ("Ögr. sayısı = %d", n);

return 0

}

int N;
N = 0

İklendirme

* static int N;

İlk değere 0 atan.

* // → Yorum satırı

* /*

çok
satır
için

*/

Format
specifiers

%d, %c, %f, %s

(İlk harfleri)

(Form) Tip değişken

Variables

char, int, float, double

1 byte

başlarına 'unsigned' gelebilir.

• short int) 2 byte

• long int) 8 byte

Örn: int i; j; k;



= assign

!= not equal

== comparison

Alg sorusu
girişlik

• Temelleri hesaplamaya bilgi

* iyi oku değeri
anla

* ANALİZ

• Belki minik bir kod

Sınav 7 haftadan

• Verimli işlem yap.

7. hafta

Data Types

Kullanıcı ortaya atar.

① void

② Scalar Types

③ Aggregate Types

Arithmetic types

Pointer, Adres saklayan tip

Integral types

Floating types

char double short signed
int enum long unsigned
float

boyutu, ayrılacak yer söylenir

Type	Size	Value Range
int	4	-2^{31} to $2^{31}-1$
short int	2	-2^{15} to $2^{15}-1$
long int	4	-2^{31} to $2^{31}-1$
unsigned short int	2	0 to $2^{16}-1$
unsigned long int	4	0 to $2^{32}-1$
signed char	1	-2^7 to 2^7-1
unsigned char	1	0 to 2^8-1

* unsigned char a,b,c

c = a + b

* unsig. char a,b,c

scanf ("%c", &a)

printf ("%c", a)

Mixing Types

→ Kendin

— Explicit Conversion (Typecasting)

— Implicit Conversion → Kurallar

int a, b, c;

float x, y, z;

a=3, b=2;

x=3, y=2;

z = x / y → 1.5 verir.

c = a / b → 1 değeri verir. ($1.5 \approx 1$)

* $z = a / y \rightarrow 1.5$ verir. → int

float 3 / 2.0

b = a / y → 1 verir

float
* $x = a / b$ → int

float sonuç için

⇒ $x = \text{(float)} a / b \rightarrow 3.0 / 2 = \underline{1.5}$
a → float olur

* İşlem bitince a hala integerdır.

* Typecasting
işlemden geçicidir.

Control Flow

if (expression) → 0 değilse doğru kalırsa gider.

statement;

0'sa yanlış

else

statement;

e.g: if (5) → direkt doğruya girer.
...



Comparison Expressions

$<, >, <=, >=, ==, !=$

Ak. di. da

↑ kullanılabılır.

* $if(a < b) \rightarrow a < b$ ise 1, değilse 0 gelir. Kollarda gider.

```
* if (x < y) {
    x = y;
    x = x + 1;
}
```

* Birden fazla ifade varsa lazım.

Kaymazsam sadece 1 iş else/ifs alınır.

```
else {
    z < y;
    z = x;
}
```

* if (x)

stat.1; $\rightarrow$ if $x \neq 0$

else

stat.2; $\rightarrow$ if $x = 0$

stat.3; $\rightarrow$ Always

Nested if's

```
int min(int a, int b, int c) {
```

```
    if (a < b)
```

```
    {
        if (a < c) {
```

```
            return a;
```

```
        } else {
```

```
            return a;
```

```
        } else if (b < c) {
```

```
            return b;
```

```
        } else {
```

```
            return c;
```

```
    }
```

Dönerin biyile fonk.
biyi eşleşmezse
hata verir.

ne döndü
reçetinin
tipi

(void de
döndürmez)

Reversability

```
#include <stdio.h>
```

```
int main() { int t1, t2, t3, enkuvcuk;
```

```
int min(int, int, int);
```

```
printf("Enter the numbers \n");
```

```
scanf("%d %d %d", &t1, &t2, &t3);
```

```
enkuvcuk = min(t1, t2, t3);
```

```
printf("%d", enkuvcuk);
```

```
return 0;
```

```
}
```

Loops

(x ve y tanımlı)
e.g. while (x < y) {

① while (expression)

statement

x = x + 1
y = y - 1

kasul

② do...while

do {

x = x + 1;
y = y - 1;

for

↑ kore yap,

kasul sağlanırsa tekrar et.

} while (x < y);

③ for (exp1; exp2; exp3)

statement

ilklendirme

değer değişimi

kullan, arttır.

```
for (i = 0, i < n, i = i + 1) {
```

```
x = x * i;  
y = x + y;
```

```
}
```

↑
i++
i--
--i
i++
↓
Arttır, kullan

* "Expression" lar da birden fazla şey yapılabilir.

Arttır, kullan



* İyi durumda iklendirebilirsiniz.

İçeride yaparsan değışken for'a özgü.

Arithmetic Operators

+, -, *, /, %

Variable Declaration

• int a, b, c;

• float x;

x = 0.0;

• double y = 0, z;

• char harf, m;

harf = 'i';

Arrays

int a[100]; // a[0] a[99]

float x[50]; // x[0] x[49]

--scanf ("%d", &N)

for (i = 0; i < n; i++) {

scanf ("%d", &a[i]);

}

for (i = 0; i < n; i++)

printf ("%d", x[i]);

}

Logical Operators

AND &&

OR ||

x < y < z

↓

if ((x < y) && (y < z))

if ((x < 0) || (x > 0))

⋮

Her şeyi bağlama

bütün Her eleman farklı

Verilen dizide elemanların "unique" olup olmadığını kontrol et. dizi

Start

Dizinin eleman sayısını giriniz.

N

1. Her elemanı sonrakilerle kontrol et.

Diziyi hep yop.

$I \leftarrow 1$ / 1
N

"Eleman giriniz."

dizi[I]

Döngüde kontrol ettiğin değeri değiştirmes.

$I \leftarrow 1$ / 0
N-1

2

$K \leftarrow I + 1$ / 1
N

F T

$N > 0$

"Eşsiz"

Stop

2

dizi[I] = dizi[K]

"Eşsiz değil"

N = -1

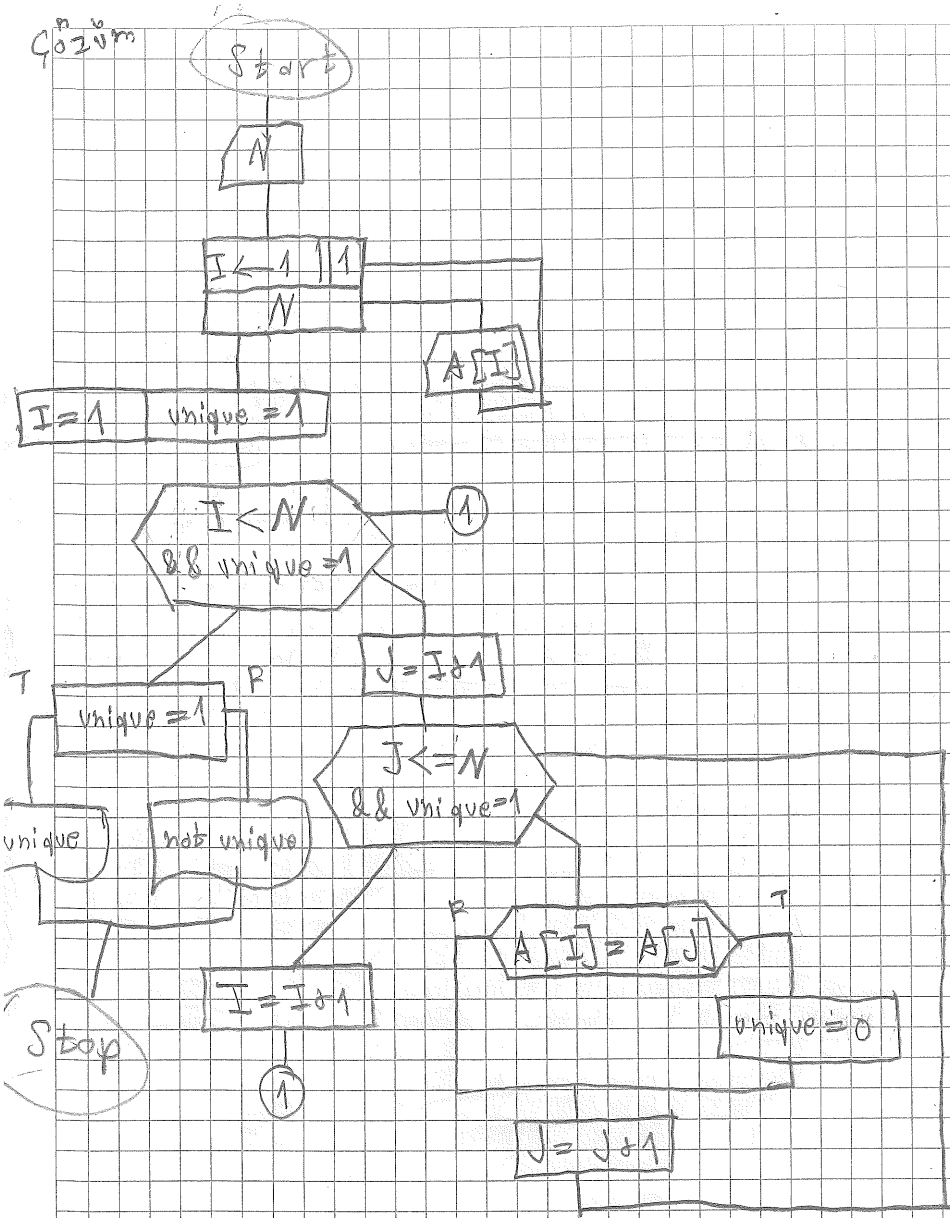
Analiz

I K N

A[I]

A[K]

Çözüm



Karakterler tek tek depolanır,

En sona null gelir,

1n

10

Strings

- Array of characters

9. hafta

Characters → Letter, number, symbol, punctuation, ^{special} chars

char a;
a = 'a';

Character
constant

* char b;
b = 50;

yazar
sayıyı

printf ("%d", b)

50'nin ascii'de denk geldiği
char.ı yazar.

printf ("%c", b)

char m;

m = ' ' / m = '.' / m = '!' / m = '0' / m = 0;

Null character

" kelime \0" null char.

scanf ("%c", &b)

"K" "E" "L" "i" "M" "E" "\0"

* char word[50]; karakter sırası

printf ("Enter the name");

Aynı

scanf ("%s", word);

// scanf ("%s", &word[0]);

a | h | m | e | \0

ilk harfi okuyup yere yazıyor
sonra takip eder.

printf ("%s", word)

* %s olmasa uzunluğu alıp for'da yaparsın,

char $\rightarrow$ '~'

string $\rightarrow$ "~~~~~"

Soru

* scanf $\rightarrow$ başlangıç kadar okur.

Çözüm

gets(word);

#include <stdlib.h>

okuy gets(word);
yaz. puts(word); } // dahil her karakteri okur.

$\rightarrow$ * printf $\rightarrow$ Null char. görene kadar yazar.

ASCII Table

'0' '9'

'A' 'Z'

'a' 'z'



Kendi içinde aradık

'Z' - 'A' + 1 $\leftrightarrow$ tüm harf var.

* Sayıda rakamlar

aynı mı?

MOD'lararak yapabilirsin.

$$'x' - '0' = x$$

1) n rakamlı bir sayı olan, dizisindeki değeri tek bir tam sayıya değiştiren algı ok. di. yaz.

Start

1 2 3 4 → 1 2 3 4

N

For'u tersten
götürmeyi öğrendik.

1

I ← 1
N

I: karakteri gir

char [I]

Kuvvet
hesaplamayı
uzatıyoruz.

toplam = 0

*

I ← 1
N

1

kuvvet = 1

toplam

J ← 1
N - I

kuvvet = kuvvet * 10

Stop

toplam = toplam + (char [I] - '0') * kuvvet

1

2

for'a tersten
giz.

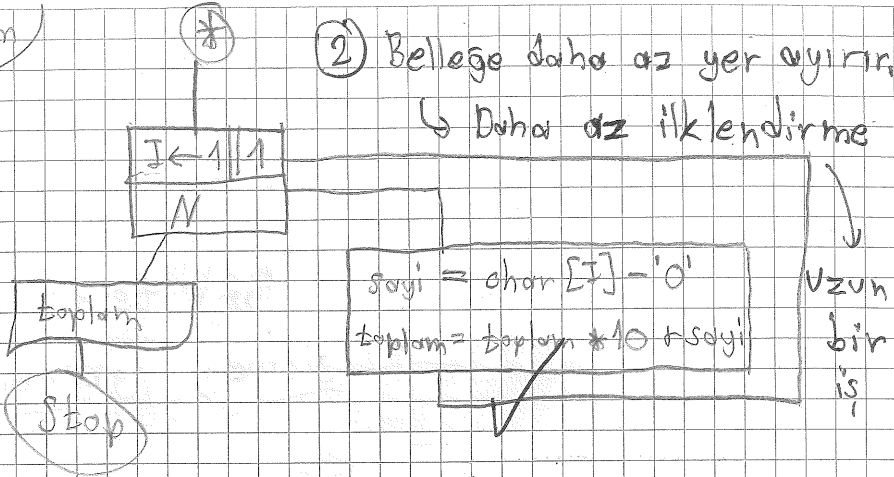
kurret';
gıkardık

① Çarpma daha az.

② Belleğe daha az yer ayırın.
↳ Daha az ilklendirme

Çözüm

③



Analiz Pongası } toplam = 1 → 10 = 12 → 120 = 123
+2 +3

```
# include <stdio.h>
```

```
# include <stdlib.h>
```

```
int main() {
    char kelime[100], word[100];
    int i;
    printf("Kelime girin: \n");
    scanf("%s", kelime);
    i = 0;
    while (kelime[i]) {
        word[i] = (kelime[i] - 'a') + 'A';
        i = i + 1;
    }
    printf("Yeni kelime: %s", word);
    return 0;
}
```

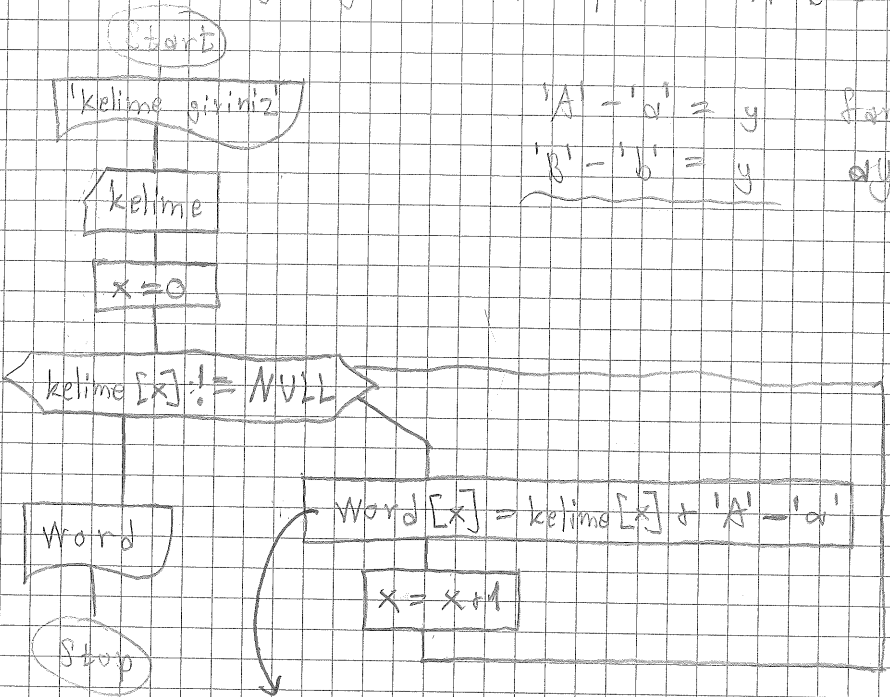
string diye bir
tip yok.



uppercase()

a b c d
↓ ↓ ↓ ↓
A B C D

II Küçük harfli kelimeyi büyük harflere çevir.



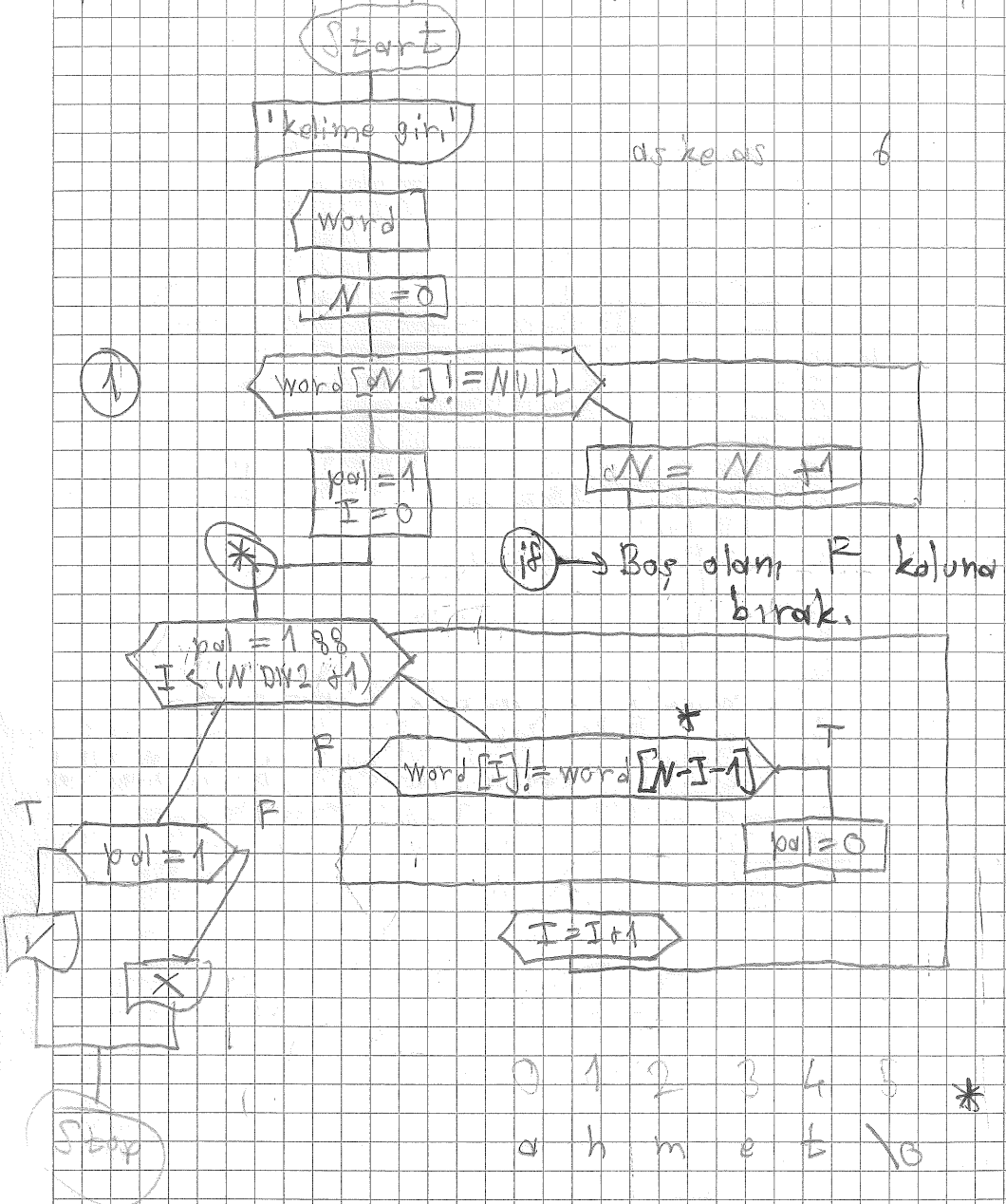
'A' - 'a' = y
'B' - 'b' = y
farklar aynı

$$\text{word}[x] = (\text{kelime}[x] - 'a') + 'A'$$

keşinai harf? büyük olanı bul.

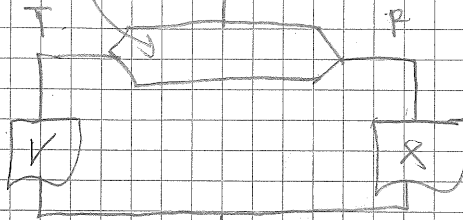
Koda

Verilen kelimenin palindrom olup olm. belirle. örneğin: akları, gızi.



Genelde tam sayılarla
modelle.

1 $I < (N \text{ DIV } 2 + 1) \text{ AND } kelime[I] = kelime[N - I - 1]$

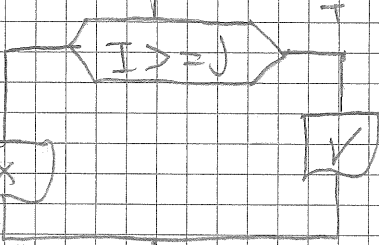


$I = I + 1$

Stop

2 $I = 0$
 $J = N - 1$

$I < J \text{ AND } kelime[I] = kelime[J]$



$I = I + 1$
 $J = J - 1$

C' 'de $\{ \}$ ile bir
dizi yapabilirsin.

char kelime[100] = "kabak"; ✓

char kelime[100];
word = 'abba'; ✗

(öğr. lavr.)

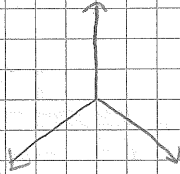
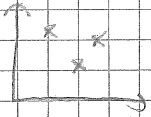
- Matris, aranan dizi içinde yapılan işlemler

Multi-Dimensional Arrays

2D → Matrix

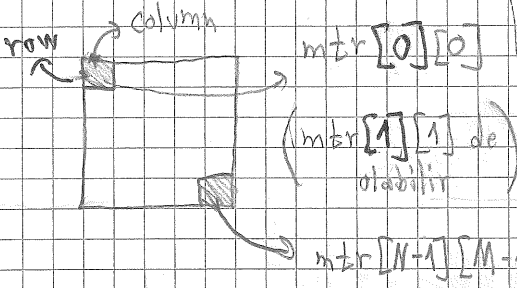
3D → Cube

4D ... N D

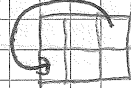
↓
Renklendirebilirsin.

2-D Arrays (Matrix)

* Dizi'de aynı türde değerleri saklar.



* int mtr[2][3];



```

for (i=0; i<N; i++) {
    for (j=0; j<M; j++) {
        scanf("%d", &mtr[i][j]);
        printf("%d", mtr[i][j]);
    }
    *Çıktı dikdö. değil small
    printf("\n")
}

```

Adress	Value
mtr[0][0]	
mtr[0][1]	
mtr[0][2]	
mtr[1][0]	

→ Bu kısım her satır sonuna \n koyar. Matris formu alır.

Verilen matrisin satır deę. lerini toplayıp yeni diziye saklayan algorit. yaz. Start

'Satır ve sütun sayı gir'

N, M

5	3	6	8
7	2	4	2
8	1	7	1
9	0	3	0

Ben

$K \leftarrow 0 // 1$
 $M-1$

toplam $[K] = 0$

$I \leftarrow 0 // 1$
 $N-1$

$J \leftarrow 0 // 1$
 $M-1$

'I', 'satır', 'J', 'sütun elemanı gir'

$k \leftarrow 0 // 1$
 $M-1$

$matr[I][J]$

$toplam[J] = toplam[J] + matr[I][J]$

toplam $[k]$

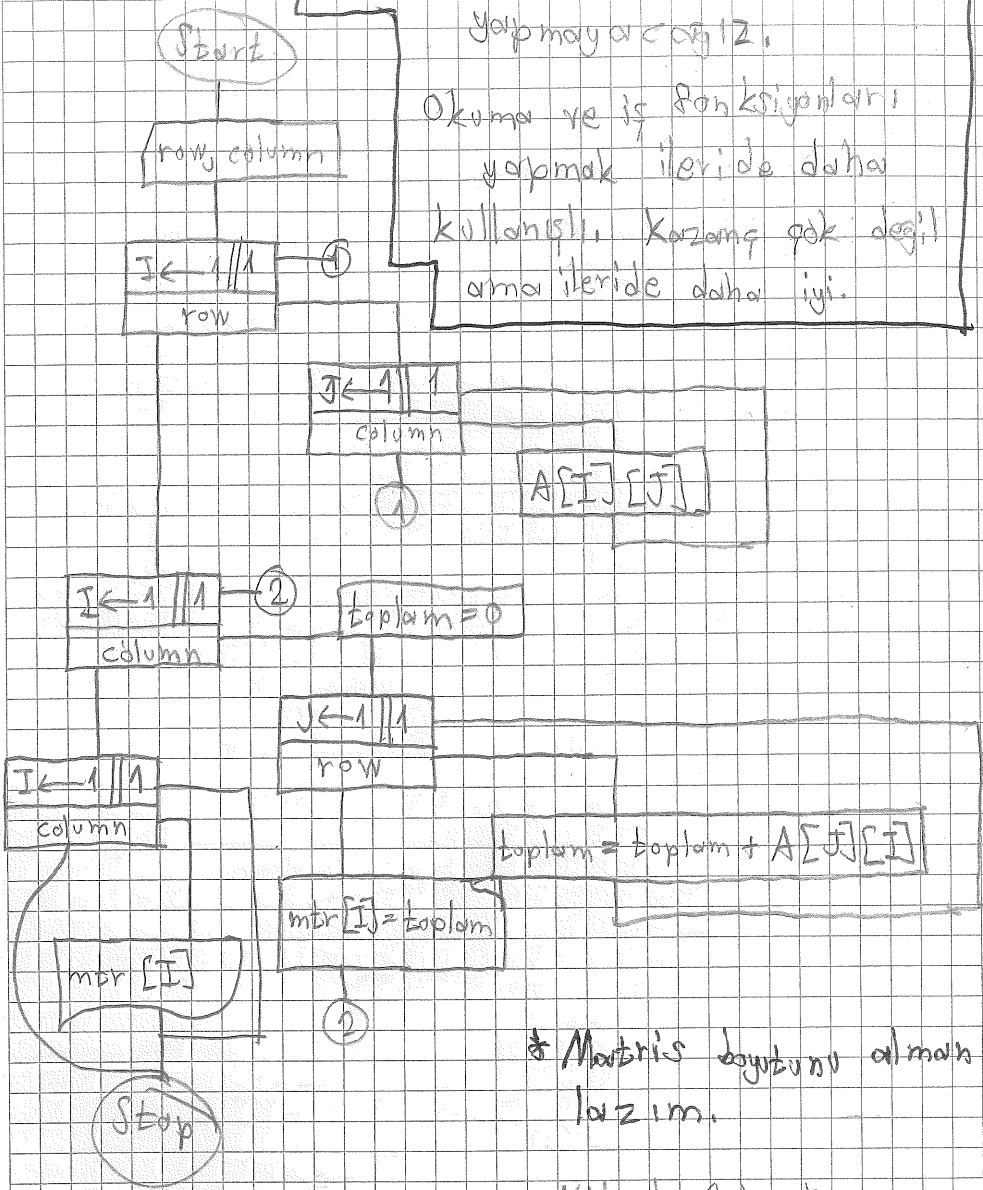
Stop

*

Girdi alırken işlem yapma yapılabilir ama bu derste böyle

Yapmayacağız.

Okuma ve iş fonksiyonları yapmak ileride daha kullanışlı. Karanlık çok değil ama ileride daha iyi.



* Matris boyutunu almak lazım.

* Video da 3 boyutlu

↳ row, column, frame

* Ses → 1 boyutlu

yapıda saklanır.

* Matris kullanımı

Görüntü işlemede pixel işleri

histogram; resimde renk parlaklık vs.
analizidir.

Ödevde yaz için
histogram yaptık.

Verilen resmin histogramını çıkararak algı akışını çiz.

* Belli değere göre
kapanıncı var.

Start

Satır sayıyı gir.

Satır ve sütun sayısını gir.

* Esit mi diye kontrole gerek yok,
İndislerle iş yapabilirsiniz.
resmi al.

row, column

I ← 1 // 1
row

J ← 1 // 1
column

I ← 0 // 1
255

image[I][J]

his[I] = 0

I ← 1 // 1
row

J ← 1 // 1
column

I ← 0 // 1
255

1

his[image[I][J]] = his[image[I][J]] + 1

his[I]

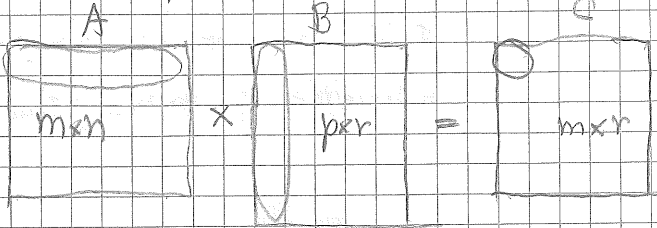
Stop

his, sayıyı dizimimiz oldu.

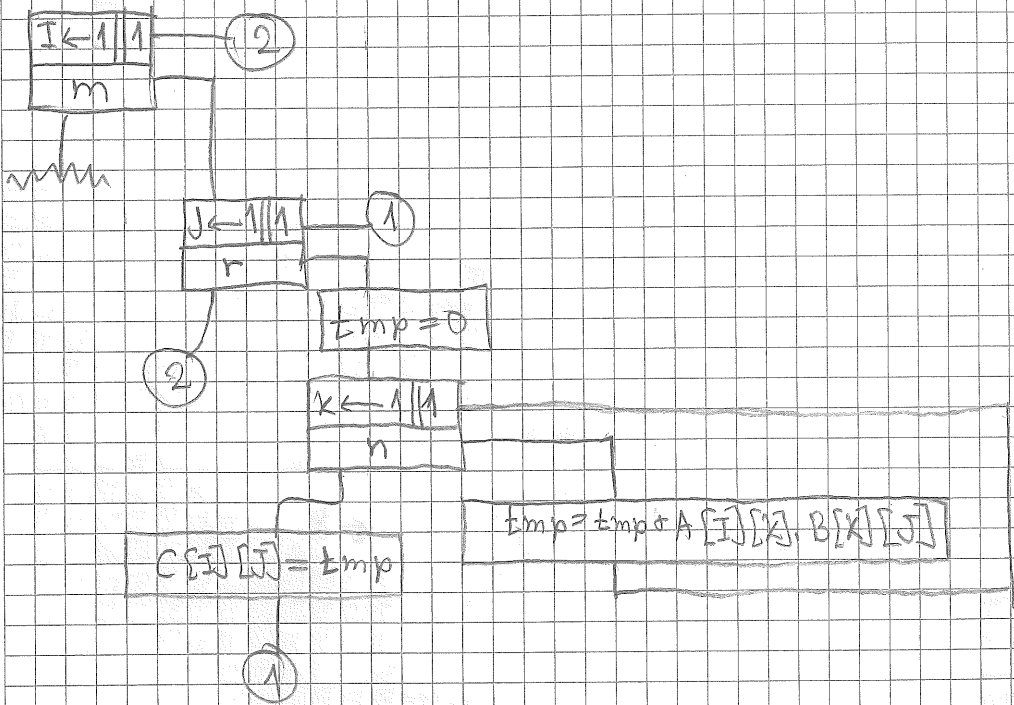
Matris Toplamı

Matris çarpımı $\Rightarrow$ En az 3 dögüde çözölün.

Matrisi çarpım alıdı, çiz, (Kendin)



$$C_{ij} = \sum_k a_{ik} \cdot b_{kj} \quad k \text{ 1'den } n=p \text{ 'ye giden.}$$



register → yazmaç

Basic Array Operations

- Traverse
- Insert
- Delete
- Update
- Min-max
- Search
- Sort

* Veriyi sıralı ve sırasız olarak alırsın,
sorted - unsorted

⑥

1 3 5 7 9 11

1 3 5 6 7 9 11

tmp = 0

tmp = tmp + A[i][k].B[i][j]

~~1 2 3~~
~~4 5 6~~
~~7 8 9~~
~~10 11 12~~

~~1 2 3~~
~~4 5 6~~
~~7 8 9~~
~~10 11 12~~

$j = 1 \rightarrow k$
 $i = 1 \rightarrow k$

Verilen N elemanı sıralı d'zeye x ekleyen alg. ak. d'i. g'z.

(Start)

I = 4
N = 6

'Eleman sayısı N giri'

I ← 1 // 1
N

'Eklenecek x' giri'

A[I]

x

yer = 1 dur = 1

dur = 1

yer ≤ N

T

x > A[yer]

F

yer = yer + 1

dur = 0

J ← N + 1

~~N~~ ← 1
J

baş ve son durumuna

A[yer] = x

A[J] = A[J-1]
~~A[J-1] = A[J-2]~~

(Stop)

XOR 13
6

1	3	5	7	9	11
1	3	5	7	9	11

Multidimensional Arrays for Strings

a	b	g	g	.
c	k			

char puzzle[5][5];

Tek for'a yazdık.

puzzle[0][0]	a	h	m	e	t	o
puzzle[1][0]	a	i	i	o		
:	r	e	i	i	o	
:	m	e	r	e	o	

for (i=0; i<N; i++)
 printf("%s", puzzle[i])
 ↑
 puzzle[i]
 Null'dan sonra baş

String matrisi
nasıl saklanır?

for (i=0; i<N; i++)
 printf("%s", puzzle[i])

* Aranım tek mi?

Aynı iş

tek indisi, önemli

* Veri sıralı mı değil mi?

Hangi yönde?

okurken başa bakar, sonrayı getirir.
 puzzle[i][0] da olur.

1 Search Algorithms

- * Aksi söylenmedikçe sıralı → küçükten büyüğe
- * " " " 1 tane vardır, anı sıra.

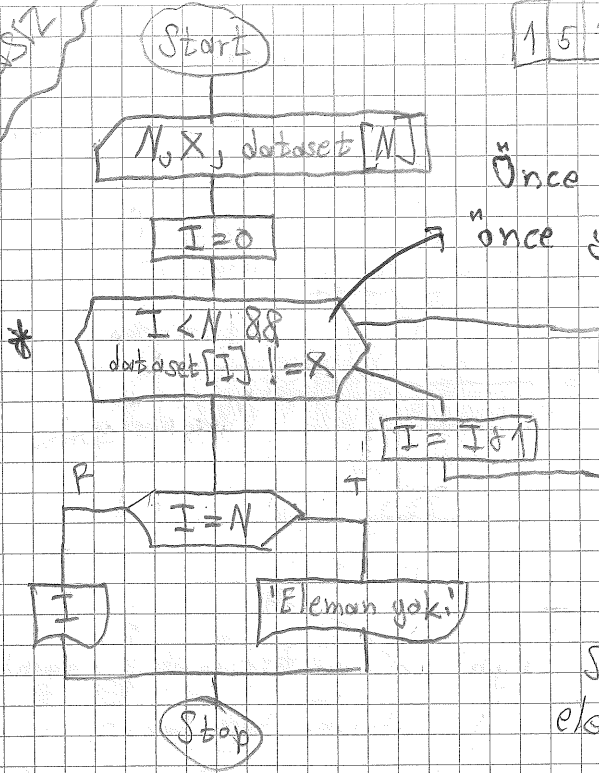
→ while ile bak, sırasızda

* Linear Search (Sequential Search)

$O(n)$

Sıraüstü

0 1 2 3 4
1 5 3 2 4
N=5



Önce yazdığın kontrol
önce yapılır.

* Analizinde
her durumu de-
ğil, farklı ve özel
durumları yap.

Sıralıda aradığın
elemandan büyüklerine
bakma.

③

* Hash, hashing, hash function

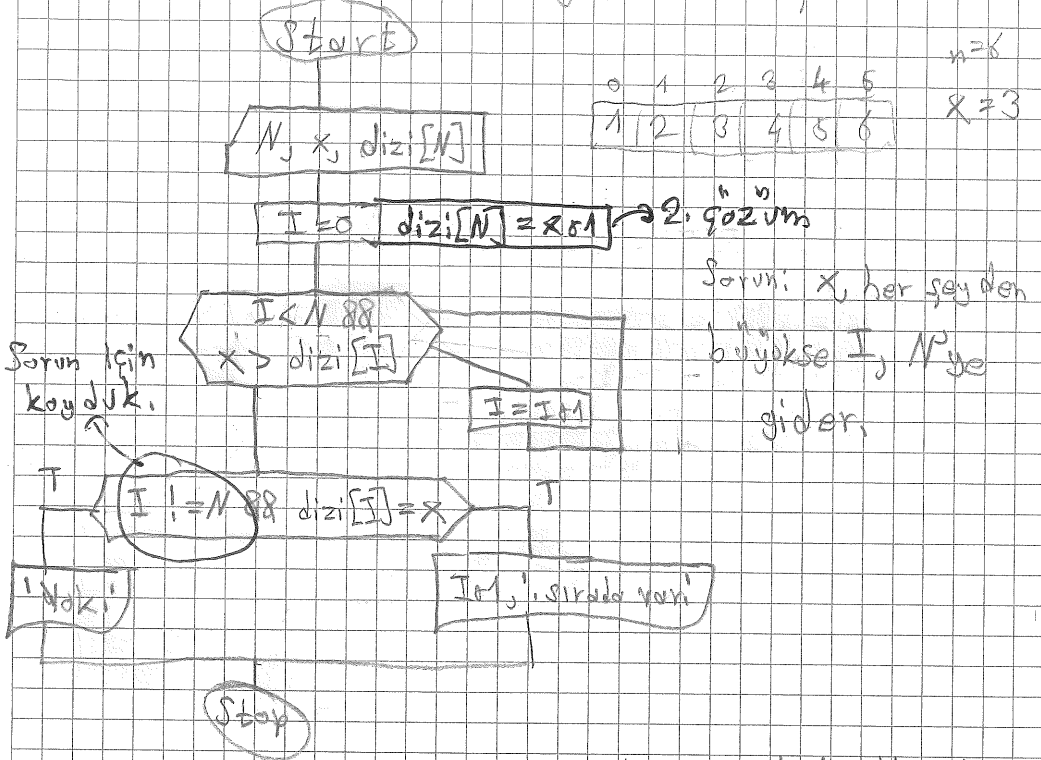
- Ders için erken.

in dislerde iş yap.

N

1) var 2) yok 3) baş 4) son
dene.

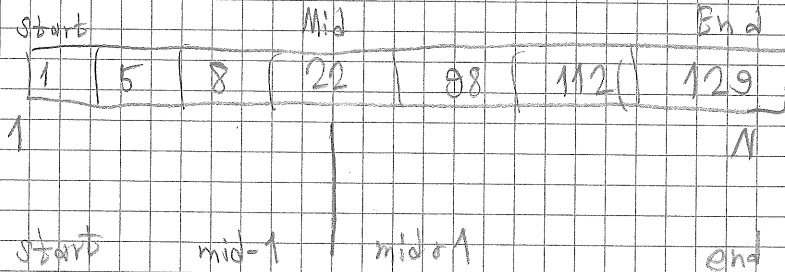
N N e l l i s i r a l d i z i n e x i a r a y a n d i . a k d i . g i z . L i n . S e r . i l e



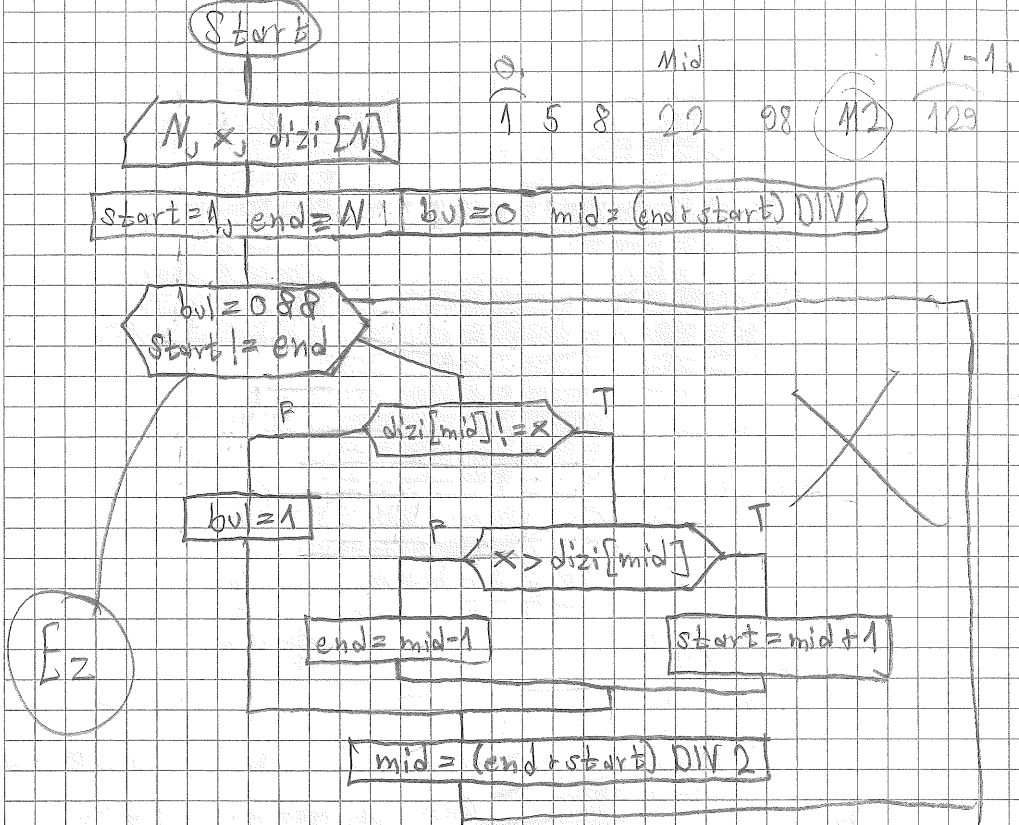
Ortalardan bul. Her turda aramayı yarıya indir.

② Binary Search Algorithm

• $O(\log_2 N)$
Genelde 2



11 Binary search yaparız a. ok. a. g. z.



Çözüm

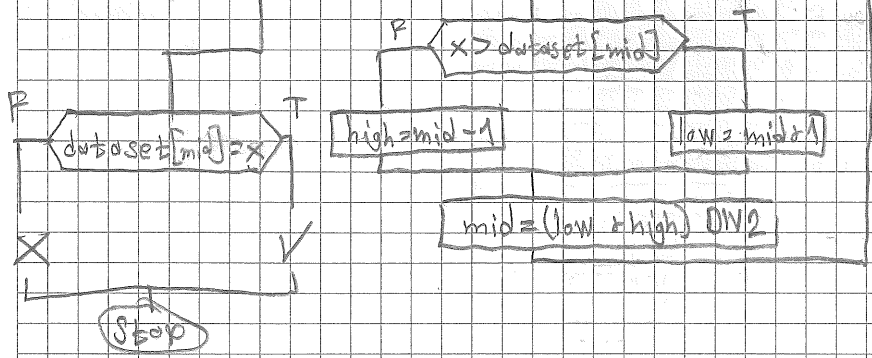
Start

$N \leq \text{dataset}[N]$

$\text{low} = 1$ $\text{high} = N$ $\text{mid} = (\text{low} + \text{high}) / 2$

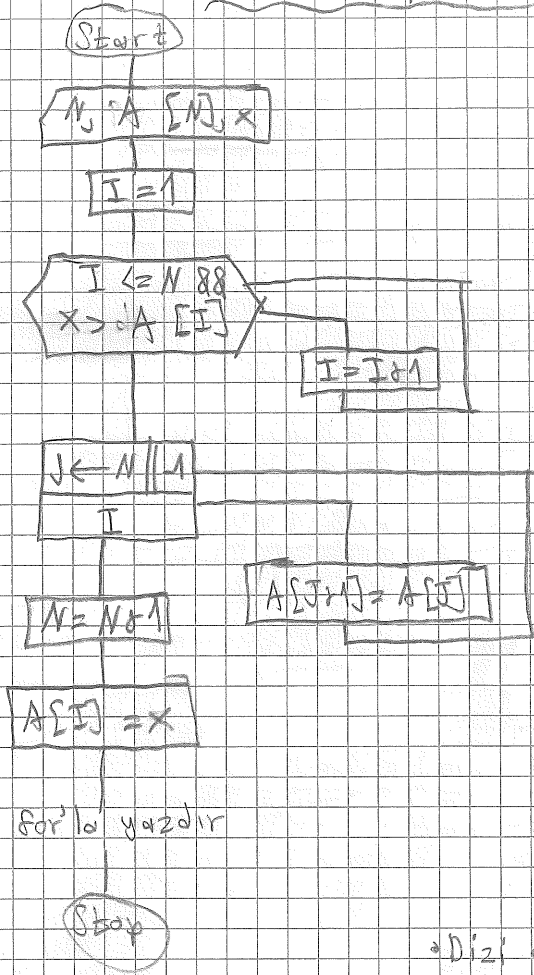
durumunu düşün.

$\text{low} \leq \text{high} \ \&\& \ \text{dataset}[\text{mid}] \neq R$



Insert Operation

Anket
online



İşi hafife alma.
Derslere gir.

*Dizi aynı zip veri tutar.

- // Projene commentler koy. Açıklan
- // 1) Temel yapı 2) Diğerlerini yapı üstüne ekle.
- // Anlatılmayan şeyleri sen yorumla.
- // Görsellik senin yaratıcılığında
- // Yılan kendine görürse ölmesi



Sırasız $\leftarrow$ Silme $\rightarrow$ Sıralı

① Silmişin yeri çeyin
ikonik bir şeye.

② Sonrakileri bir yere
kaydırıp sil.

Sıralama Algoritmaları

Sort

* Sayılar, karakterler, stringler $O(n^2)$

$\rightarrow$ Bubble Sort

$\rightarrow$ Selection Sort

$\rightarrow$ Insertion Sort $O(n^2)$

iki karşılaştır.

kagit sıralar gibi

1 2 8 7 6 5 4

$O(n^2)$

$N-1$

1 2 7 8 6 5 4

1 2 7 6 8 5 4

1 2 7 6 5 8 4

1 2 7 6 5 4 8

N^2

Toplam

1 2 7 6 5 4 8

1 2 7 6 5 4 8

1 2 6 7 5 4 8

1 2 6 5 7 4 8

1 2 6 5 4 7 8

2.

kontrol

$N-2$

...

* Küçük veride elverişli

* Çok fazla swap yapılır.

(2'li yer değiştirme)

Belleğe çok fazla erişim.

* Her elemanı yerine kaydır.

Sağdan başlayarak tek tek
sola kaydır.

Örne büyük olduğu ça kaydır.

1 3 5 7 4 2 6 2. eleman başla.

GİZ. Küçük
veride iyi
çalışır.

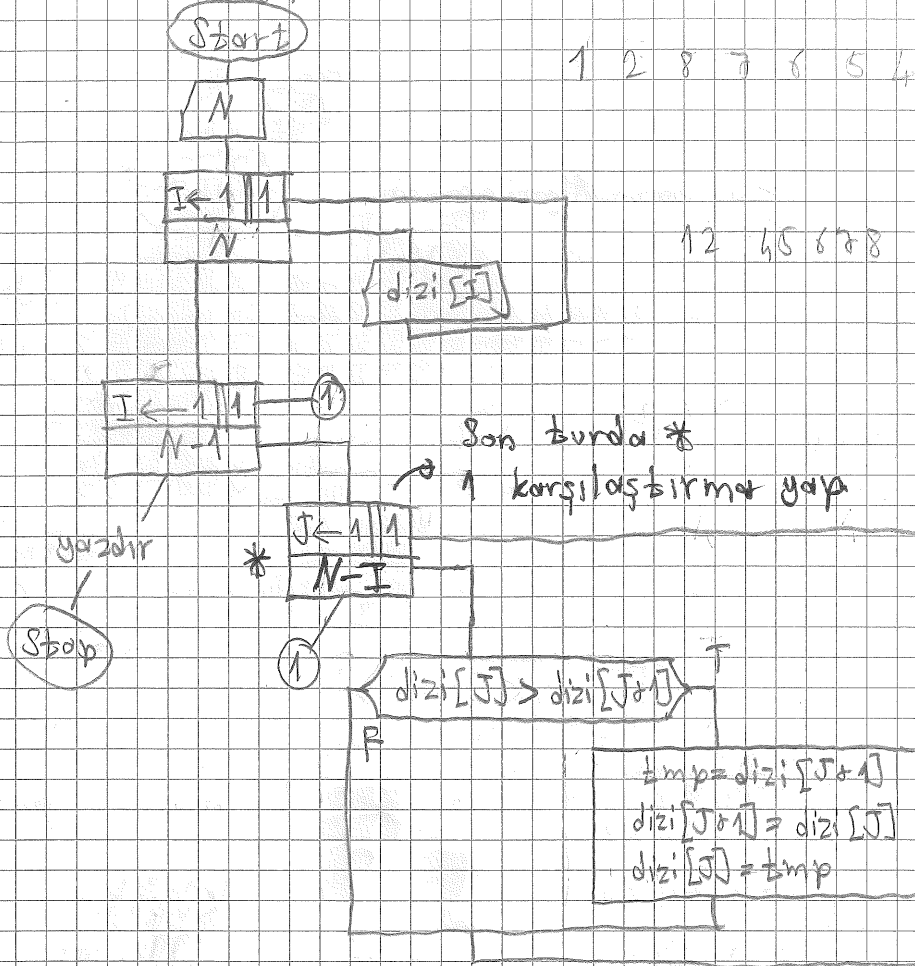
* Sürekli swap yapılmaz.

sayıyı temp'e al, yerini bul,

diğerlerini kaydır, en son kay.

4 1 2 3

// Bubble Sort yapmam ol. ek. di. g'iz.



* İşlem yapılmadıysa bitir.

İyileştirme: Sıra olduktan sonra artık bakmana gerek yok. while ile

YAP

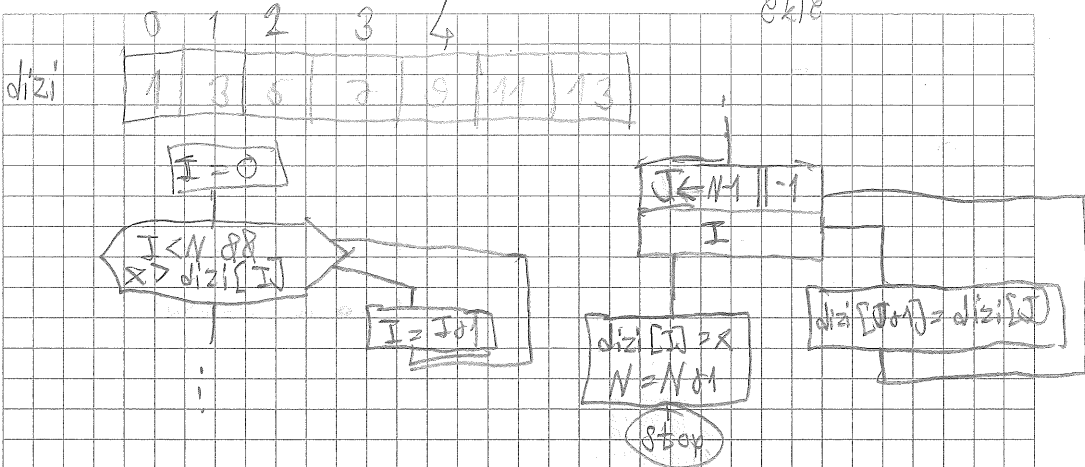


DOĞUŞ
ÜNİVERSİTESİ

while
for dogus.edu.tr | 444 79 97 | info@dogus.edu.tr

L...

$x \geq 8$ ekle



Alan matris dışına çıkıyorsa matris sınırına kadar bak.

Bazı İst.
* Sorusu Tüm alanı → Alan için for'da kapsama alanını bırak, 0 değilse göster. taranması gereksiz.

* Soru mu while mı kullanacağın önemli.

* Verimin önemli

Bildikçe böyle bir metin veremezsin.

* Statik ve dinamik yer ayırmayı (bellekte) öğren.

* Sembolik çalışmada alan kazanmıyorsun. Kaydırma daha mantıklı.

→ Yer açma programı çalışmadan önce yapılır.

define kullanarak yapabilirsin. #define

Oraya sayı koymam gerek. ***

* Kullanıcıdan aldığın değerle matrise yer açamazsın.

Notu 1) Syntax hatası → j uuzuma gibi

2) Mantıksal hatalar → 0'a bölme, sonsuz döngü gibi

Düşünce önemli. *

Selection Sort

Her turda en küçük elemanı seç, başa koy.

1 2 8 7 6 5 4

1. tur
1 2 8 7 6 5 4
1 2 8 7 6 5 4
1 2 4 7 6 5 8

N-1

İlk k elemanın
sırala derse:

...

I ← 1
N-1

k yaz.

Stop

min = I

J ← I + 1
N

dizi[J] < dizi[min]

min = J

tmp = dizi[J]
dizi[J] = dizi[min]
dizi[min] = tmp

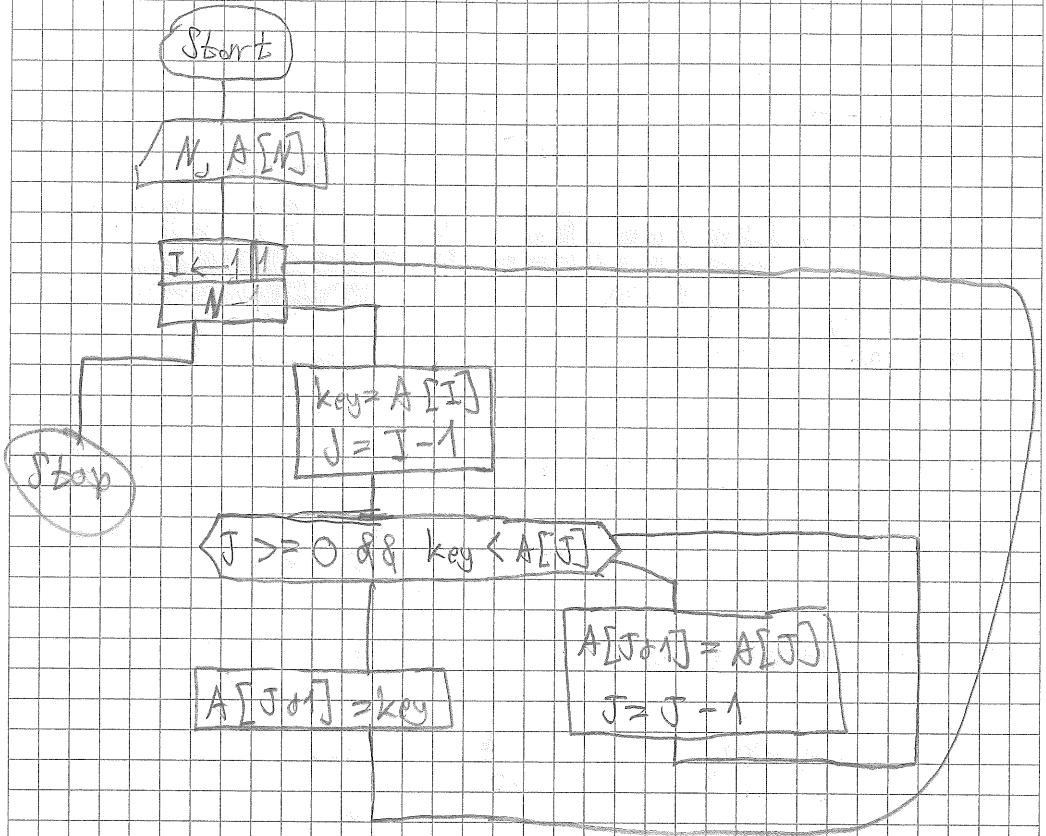
N=4

I	min	J	dizi[J]	dizi[min]	tmp
1	1	2	8	9	
2	3	7	8	8	
3	4	6	7	7	6 8 7 9
4					
2	2	3	7	8	

1 2 3 4
9 8 7 6



Insertion Sort



0	1	2	3	4	5
9	5	7	3	2	6

N=6

1. bur	I=1	J=0	key=9	5	9	7	3	2	6
2. bur	I=2	J=1	key=5	5	7	9	3	2	6
3. bur	I=3	J=2	key=7	3	5	7	9	2	6
		J=1		2	3	5	7	9	6
		J=0		2	3	5	6	7	9

4. bur I=4 J=3 key=2

TAM DEĞİL → controls operasyon

İşletim Sistemleri

- Windows, Linux, MacOS,

* İşletim sistemleri 2. bellekte bulunur

- Batch Job

→ Batch processing

→ Komut ver, sonucu bekle.

İş zamanlı

① Interactive processing

② Batch processing

Sıralı

- Queue → FIFO

* Time-sharing

İş zamanlı çalışma

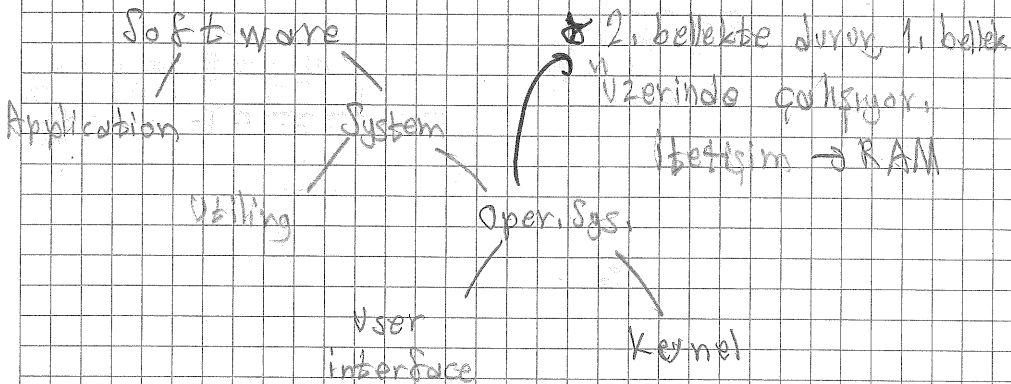
- Gömülü sistemler

→ Belirli görevleri olur.

İş. Sist. Görevleri

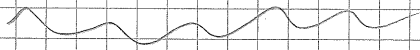
① İşlem ② Bellek ③ Dosya ④ Network } yönetimi

⑤ Interrupt Handling → Öncelik sırası verme



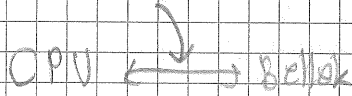
- Boot loader

* Booting Process



* Communicating with Osh. Dev.

Controller ile



• Wired Commu.

• Wireless Co.

↳ GSM, 2G, 3G, NFC, Wi-Fi

* Network Classification

PAN, LAN, MAN, WAN

→ PC'ler aralarında 0 kadar sinyal gönderilir.

Çözümü Router [Sinyal artırır] / Bridge [Benzer]

Switch [ağları bağlar.]

* IP → 32 bit

192, 168, 1, 1 → 0-255

* Internet Applications

• IPv6 → 128 bit hepsi

• File Transfer pr. (FTP)

• telnet • HTTP • SMTP

1G → Analog

sinyallerle aktarılır

2G → Dijital.

Insertion Sort

Start

$N, A[N]$

$I \leftarrow 1$
 N

$temp \leftarrow A[I]$
 $J \leftarrow I - 1$

0	1	2	3
2	4	5	3
		4	5

Stop

$J > 0 \text{ \&\& } temp < A[J]$

$A[J+1] = temp$

$A[J+1] = A[J]$
 $J = J - 1$

$I \leftarrow 0$
 $N-1$

$A[I] = 0$

Stop

$J = I + 1, say = 0$

$J < N \text{ \&\& } diz[J] = 0$

$J = J + 1, say = say + 1$

$K \leftarrow J$
 $N-1$

$A[K - say] = A[K]$



DOĞUŞ
ÜNİVERSİTESİ

dogus.edu.tr

444 79 97

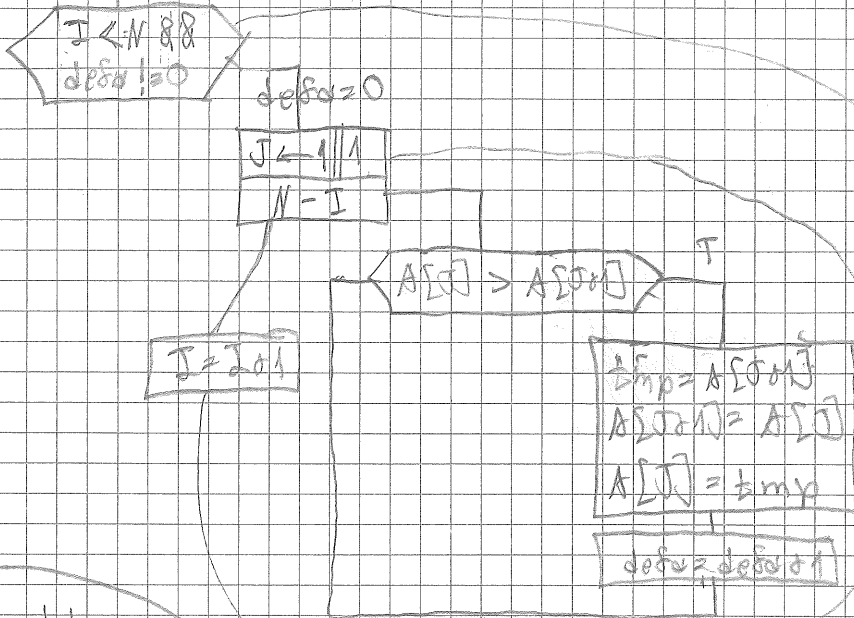
info@dogus.edu.tr

Sortlar

Bubble

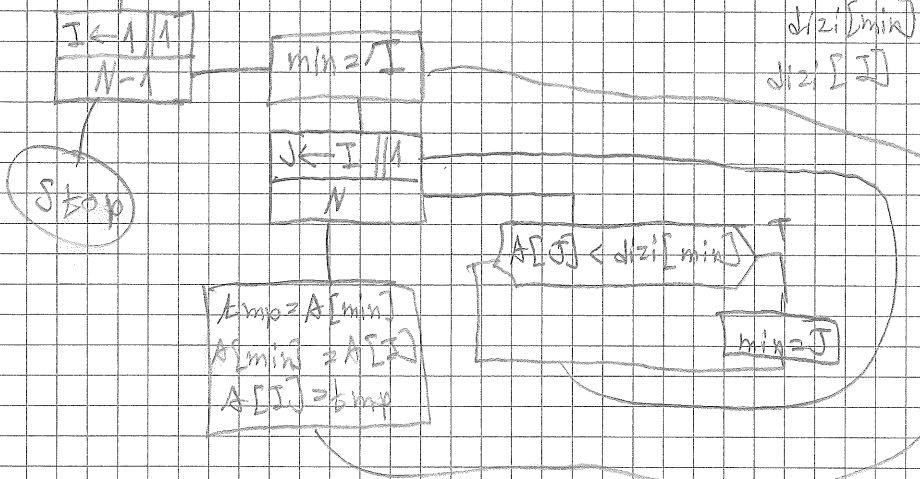
$N, A[N]$ $defsu=1$

8 7 1 6 2 3 4



Selection

$N, A[N]$



Gezgin satıcı problemi

Algoritma için

chatgpt

Sort

Bubble : N^2

Merge : $N \log N$

Quick : $N \log N$

↘ if reverse : N^2
ordered

Robert Sedgwick
- Algorithms